



TOPIC 1

INTRODUZIONE

- Ogni **progetto informatico** è sicuramente **incompleto** fino a quando non viene corredato da una documentazione esauriente.
- In un progetto informatico la **documentazione** è presente in tutte le fasi, dalla raccolta dei requisiti, passando per la documentazione di analisi e tecnica di implementazione, fino ad arrivare alla documentazione per l'utente finale.



- Quali **documenti** è opportuno generare in un progetto ?
- La **documentazione** di progetto va generata all'inizio o alla fine ?
- Come tenere sempre **aggiornata** la documentazione quando i requisiti o le implementazioni cambiano ?
- Che **standard** usare per creare una **buona documentazione** di un progetto informatico ?
- C'è stata un'ampia diffusione delle metodologie **Agili**. E' vero che non è prevista documentazione ?
-



- **Definizione** di funzioni, vincoli, prestazioni, interfacce e qualsiasi altra **caratteristica** che il sistema dovrà possedere per soddisfare le necessità del cliente
- Redazione di un **Documento di Specifica dei Requisiti Software**, che sia completo, preciso, consistente, non ambiguo, comprensibile sia al committente che allo sviluppatore
- Predisposizione di un **piano** di test e della versione «0» (zero) del manual utente

N.B. Definizione di COSA il sistema dovrà fare senza descrivere COME



- I **desideri/bisogni** di un cliente possono essere espressi come **scopi**, ovvero come **obiettivi di un business**
- Esempio: voglio viaggiare, mi serve di pianificare un tragitto in treno e comprare il biglietto
- Nel **progettare** un sistema che permetta di conseguire uno scopo dovremo specificare una serie di funzioni
- La relazione tra scopi e funzioni che li conseguono **è l'oggetto dell'analisi dei requisiti**



- Un **SRS** (documento di Specifica dei Requisiti Software) costituisce il punto di **convergenza** di tre diversi punti di vista: cliente, utente, sviluppatore
- Un **SRS** fornisce un punto di riferimento per la **validazione del prodotto finale**; un **SRS** di qualità è il pre-requisito per un software di alta qualità e riduce i costi di sviluppo
- Un errore nell'**SRS** produrrà **errori** nel sistema finale
- Correggere un errore dell'**SRS** dopo lo sviluppo costa in media 100/200 volte più che correggerlo durante la fase di **Analisi** (dato statistico raccolto dalla bibliografia moderna e tratto dall'esperienza)



- La parte più **complessa** nella realizzazione di un sistema software consiste nel decidere cosa realizzare.
- Nessuna parte del lavoro pregiudica maggiormente il risultato se viene eseguita in modo errato.
- Nessuna altra parte è più difficile da correggere successivamente.



- **Requisito:** una caratteristica che il sistema deve possedere per realizzare lo scopo del sistema stesso
- Il processo di ricerca, analisi e documentazione dei requisiti è chiamato **Ingegneria dei Requisiti** (Requirements Engineering)
- Per l'Institute of Electrical and Electronic Engineering (IEEE), i requisiti:
 - Esprimono **capacità** e **condizioni** (**vincoli**) necessarie per risolvere problemi o realizzare obiettivi di business degli utenti
 - Sono documentati da contratto, **standard**, specifiche o altri documenti formalmente convenuti che descrivono le «Capacità e Condizioni»
 - La loro documentazione, in genere, si basa su una rappresentazione grafica delle Capacità e delle Condizioni che descrivono

- **Definizione dei requisiti:** Descrizione **orientata al cliente** delle funzioni del sistema e delle restrizioni sulle sue operazioni
- **Specifica dei requisiti:** Descrizione **dettagliata e precisa** della funzionalità del sistema e delle restrizioni; intesa per comunicare cosa è richiesto dallo sviluppatore e per servire da base di un contratto per lo sviluppo del sistema



TOPIC 2

I REQUISITI E GLI STAKEHOLDER

- Stabilire **cosa richiede il cliente** ad un sistema software (scopo) ...
- ... **senza definire** come il sistema verrà **costruito** (funzioni)
- Ma come definiamo i requisiti funzionali di un sistema?



Domande utili per scrivere i requisiti

- Cosa deve fare il **software**?
- Che **interfacce** ha con i suoi **utenti** (con HW, con altro SW)?
- Che **prestazioni** deve esibire il SW?
- Quali **attributi** (es. portabilità) dovrà avere?
- Quali **vincoli** dovrà soddisfare?
- ...



- I **requisiti** servono per far **comunicare correttamente**:
 - Gli **Stakeholder** (Parti Interessate) tra di loro;
 - Portatori di **conoscenza** del Dominio Applicativo con gli **Sviluppatori** (portatori di **conoscenza** del Dominio delle Tecnologie Informatiche).
- Definizione di **parte interessata**: è una classe di persone o sistemi che, a vario titolo, sono **interessate** ad una o più caratteristiche del prodotto software



Gli stakeholder

- Termine inglese: i detentori degli «**stake**», cioè i portatori di interessi riguardo al sistema in esame
- Il principale stakeholder è il **cliente**, interno o esterno
- Altri **stakeholder** sono:
 - gli sviluppatori
 - il management



Gli Stakeholder: Persone

- Il **Manager/Committente/Customer** determina i vincoli di costo e di personale da utilizzare, **requisiti** del processo di sviluppo
- L'**Utente** è interessato ai **servizi** resi dal sistema, alla loro **qualità** ed alla loro **adattabilità**
- Lo **Sviluppatore** è interessato alla **componibilità**, alla **qualità** ed alla **estensibilità** del sistema



- **Organi di Legislazione o di Controllo** interessati a comportamenti o a livelli di qualità (esempi vari: controllo della composizione degli scarichi, certificazione della qualità, ecc.)
- **Sistemi Cooperanti** con cui i prodotti da sviluppare devono comunicare

- Dichiarano, in **linguaggio naturale** e/o **corredati da diagrammi**, quali **servizi** il sistema dovrebbe fornire e i **vincoli** sotto cui deve operare
- Sono rivolti a lettori **disinteressati** ai dettagli dei servizi
- Dovrebbero specificare **SOLO** il comportamento esterno del sistema
- Il **linguaggio naturale** può portare a diversi problemi:
 - Manca di chiarezza
 - Confusione dei requisiti
 - Mescolanza dei requisiti

- Strategie:
 - Adottare un formato standard
 - Individuare univocamente i requisiti
 - desiderati
 - obbligatoria

- Definiscono le **funzioni, i servizi ed i vincoli operativi** del sistema in modo dettagliato.
- Il documento dei requisiti di sistema **dovrebbe essere preciso** e definire esattamente **cosa deve essere implementato**
- Sono definiti per i lettori che devono sapere con precisione cosa il sistema dovrà fare
- Sono «**versioni espanse**» dei requisiti utente e dovrebbero descrivere semplicemente il comportamento esterno del sistema, **non come il sistema dovrebbe essere progettato o implementato**
- Il **linguaggio naturale** è poco adatto alla descrizione dei requisiti di sistema

- **Linguaggio naturale strutturato:** Limitazione delle forme linguistiche usabili, uso di costrutti di controllo tipo linguaggi di programmazione (if-then-else, while...), uso di forms (moduli predefiniti: nome e descrizione della funzione, input, output, pre-condizioni, ecc.)
- **Linguaggi di descrizione di progettazione:** Linguaggi tipo «linguaggi di programmazione» con caratteristiche più astratte per esprimere requisiti e definire modelli operazionali. Poco usati ma comodi per definire i requisiti di interfaccia poiché definiscono i requisiti fornendo una visione operativa del sistema.
- **Notazioni grafiche:** Un linguaggio grafico, aiutato da annotazioni testuali, viene usato per definire i requisiti funzionali del sistema (es. casi d'uso proposti da UP).
- **Specifiche matematiche:** Linguaggi basati su concetti matematici (es. insiemi o automi a stati finiti). Riducono le ambiguità ma sono di difficile comprensione per il cliente.

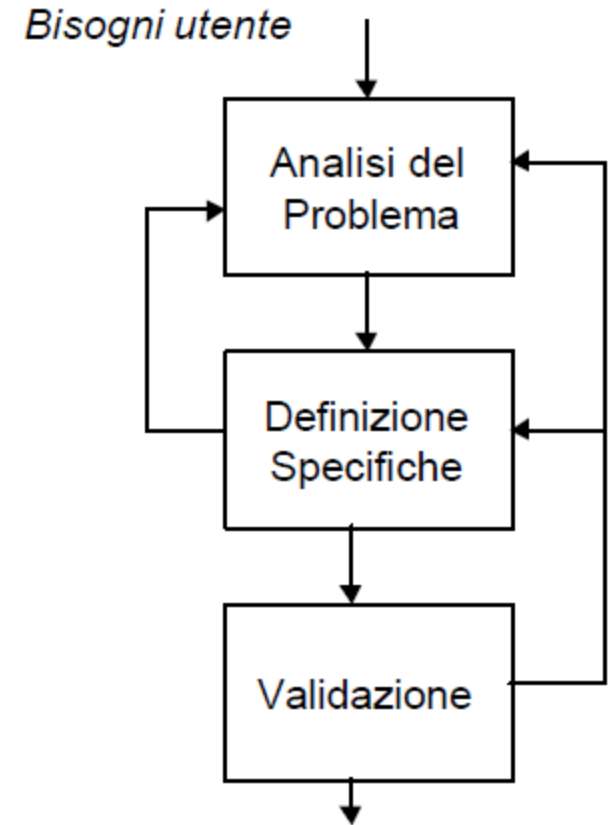
TOPIC 3

IL PROCESSO DI RACCOLTA DEI REQUISITI E GLI SRS

- **Analisi del problema:** Partendo da una generale definizione delle necessità dei potenziali utenti, comprendere: il problema, le condizioni di risoluzione ed i vincoli. Lo scopo è di **comprendere cosa deve fare** il sistema software che si vuole costruire.
- **Specificazione dei requisiti:** trasformazione dei requisiti in un documento di specifiche tecniche e funzionali caratterizzanti il sistema
- **Validazione delle specifiche:** le specifiche, formalizzate, vengono riviste con l'utente/committente per validarle

Il Processo di specifica dei Requisiti

- Processo con **feedback** fra le attività
- Durante l'analisi:
 - **decomporre** il problema in più parti e relative relazioni
 - **uso di modelli** per rappresentare le informazioni raccolte (DFD, Object, etc.)
- **Transizione da analisi a specifica:** non immediata, giacché l'**analisi** fissa la struttura del problema, mentre la **specifica** definisce il comportamento esterno del sistema



- Ottenere una chiara **comprensione** delle necessità del **committente/utente**, cosa è desiderato dal sistema Sw, quali i vincoli da assumere
- **Comprensione del dominio applicativo** dal punto di vista del **committente/utente** e del comportamento esterno
 - individuazione degli elementi che lo compongono
 - attività svolte
 - informazioni trattate
 - e delle relazioni esistenti tra essi
- Il **principio del partizionamento** può essere applicato sia rispetto agli oggetti che alle funzioni del dominio applicativo
- Può basarsi su una **rappresentazione** (modello) del dominio applicativo

- **Approccio informale**
 - nessun modello formale del sistema viene costruito
 - la raccolta di informazioni sul sistema è ottenuta grazie a **interazioni dell'analista col cliente e l'utente finale**, l'uso di questionari, studio di documentazione esistente, ...
 - si procede attraverso la **formulazione e il raffinamento di vari SRS** che sono sottoposti alla validazione nell'ambito di opportuni meeting
- Approccio basato su **modelli concettuali del problema**: produce rappresentazioni formali del problema
- Approccio basato sulla **prototipazione**: il problema viene **analizzato** ed i requisiti sono compresi grazie all'uso di un **prototipo del sistema** da parte di cliente ed utenti

- La tecnica dell'**Analisi Strutturata**
 - Basata sull'uso di **Data Flow Diagrams** e **Dizionario dei Dati**
 - L'analisi del problema viene eseguita usando l'approccio della decomposizione delle funzioni
- **Analisi Object-Oriented**
 - L'analisi del problema viene eseguita usando l'approccio della decomposizione in oggetti (entità/ concetti del dominio del problema)
 - Si avvicina molto al modo «**reale**» e agli schemi mentali delle «**persone**»
- **Modello Entità- Relazioni**
 - Usato per modellare i dati e le relative relazioni
 - Usato per progettare database

- La fase di analisi **non definisce** tutti gli aspetti del software quali:
 - Interfacce utente
 - Gestione dei casi di errore
 - Vincoli di prestazioni
 - Vincoli di progettazione
 - Vincoli di aderenza agli standard
- Pertanto la conoscenza acquisita nella fase di analisi dovrà essere tradotta nella «Specifica Dei Requisiti»
- **Ma la traduzione non è immediata !!!**
- Quali caratteristiche di qualità dovrà **possedere** un buon **SRS**?

- **Corretto**: se ogni requisito presente nell'SRS è realmente richiesto al sistema finale
- **Completo**: se ogni funzione richiesta al software ed il suo comportamento rispetto ad ogni possibile input è specificato
- **Non Ambiguo**: ogni requisito ha una sola interpretazione (formale vs. informale)
- **Verificabile**: se è possibile verificare che il sistema realizzi ogni requisito (richiede la non ambiguità dei requisiti)

- **Consistente**: se nessun requisito è in contraddizione con gli altri
- **Modificabile**: se la struttura e lo stile dell'SRS sono tali da consentire **facili modifiche**, preservando consistenza e completezza (un SRS con ridondanze non si modifica facilmente)
- **Tracciabile**:
 - Se l'origine di ciascun requisito è chiara e può essere referenziata nello sviluppo futuro (def. IEEE)
 - **forward traceability**: requisito collegabile a qualche elemento del progetto e del codice
 - **backward traceability**: dal progetto e dal codice è possibile risalire al requisito corrispondente

- **Requisiti Funzionali:** la specifica di quali output sono prodotti dal sistema rispetto a dati input e la logica (non l'algoritmo) con cui gli output sono ottenuti. Inoltre il range di input validi, il comportamento rispetto ad input/output non validi
- **Requisiti sulle Prestazioni**
 - statiche: # terminali, # utenti simultanei, # file da processare
 - dinamiche: tempi di risposta, throughput
- **Vincoli di progettazione:** standard da seguire, limiti nelle risorse hardware/ software impiegate, ambiente operativo, affidabilità, sicurezza
- **Requisiti sulle Interfacce esterne:** la specifica delle modalità con cui il software interagisce con l'utente, hardware ed altro software

- **Informali**: le specifiche del sistema sono descritte in linguaggio naturale
- **Formali**: la specifica è un oggetto formale rappresentata in una notazione definita in modo rigoroso sul piano sintattico e semantico
- **Semi-formali**: si collocano in una posizione intermedia fra i due estremi
 - utilizzano, in genere, notazioni grafiche con semantica poco formalizzata
 - sono di facile interpretazione
 - Non consentono, o almeno riducono, interpretazioni ambigue
 - Sono accompagnate da descrizioni in linguaggio naturale

N.B. I modelli semi-formali sono allo stato dell'arte i più flessibili ed i più usati

Table of contents

1.Introduction

- 1.1. Purpose
- 1.2. Scope
- 1.3. Definitions, Acronyms and Abbreviations
- 1.4. References
- 1.5. Overview

2.General Description

- 2.1. Product Perspective
- 2.2. Product Functions
- 2.3. User Characteristics
- 2.4. General Constraints
- 2.5. Assumptions and Dependencies

3.Specific Requirements

3.1.Functional Requirements

3.1.1. Functional Requirement 1

- 3.1.1.1. Introduction
- 3.1.1.2. Inputs
- 3.1.1.3. Processing
- 3.1.1.4. Output

3.1.2. Functional Requirement 2

...

3.1.3.Functional Requirement n

3.2. External Interface Requirements

- 3.2.1. User Interfaces
- 3.2.2. Hardware Interfaces
- 3.2.3. Software Interfaces
- 3.2.4. Communications Interfaces

3.3. Performances Requirements

3.4. Design Constraints

- 3.4.1 Standard Compliance
- 3.4.2. Hardware Limitation

...

3.5. Attributes

- 3.5.1. Security
- 3.5.2. Maintainability

...

3.6. Other Requirements

- 3.6.1. DATABASE
- 3.6.2. Operati
- 3.6.3. Site Adaptation

...

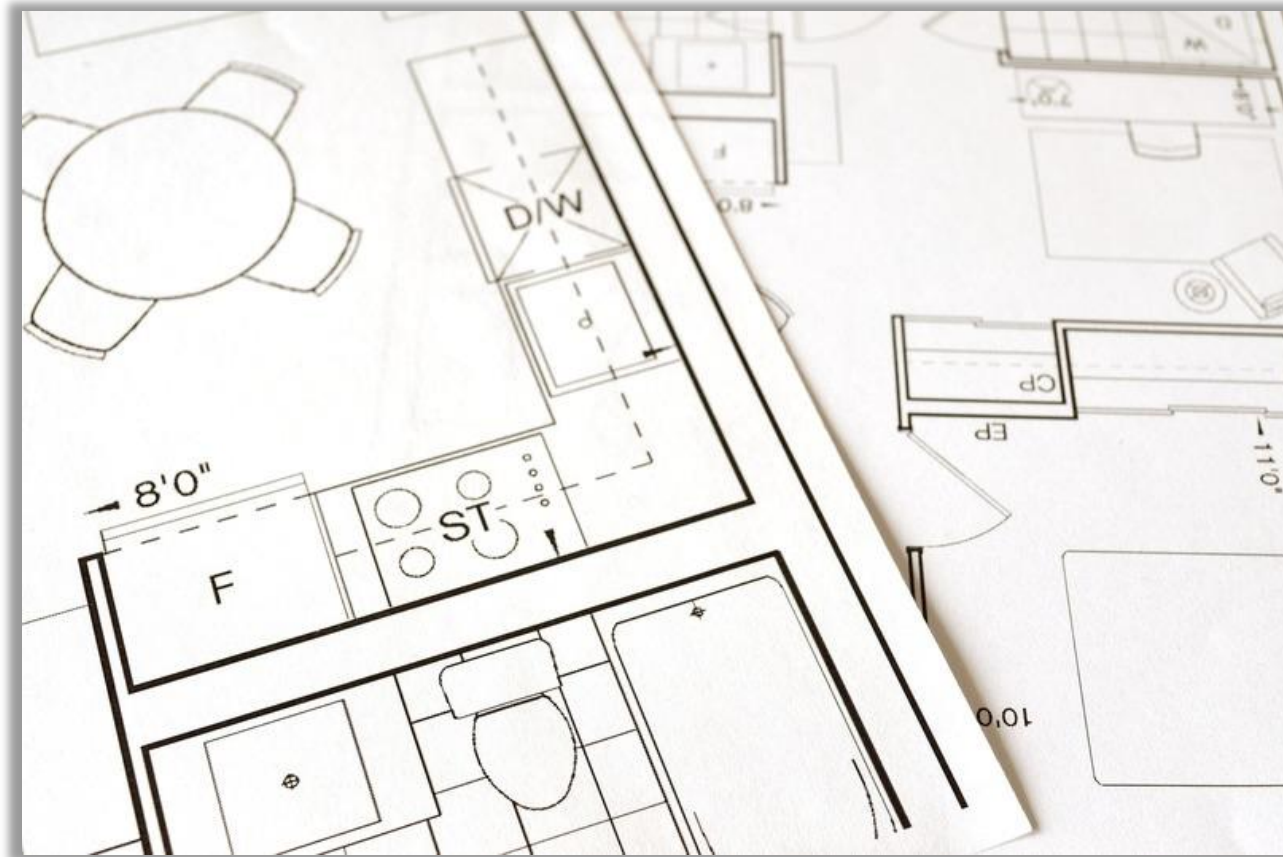
- L'obiettivo è quello di assicurare che l'SRS **rifletta accuratamente e con chiarezza** i requisiti effettivamente richiesti al software
- Tipi di **errori riscontrabili** in un SRS
 - **Omissione** (mancata presenza di un requisito)
 - **Inconsistenza** (contraddizione fra i vari requisiti o dei requisiti rispetto all'ambiente operativo)
 - **Incorrettezza** (fatti non corretti nell'SRS)
 - **Ambiguità** (requisiti con significati multipli)
- Le tecniche di validazione devono puntare su tali errori

- **Revisioni ed ispezioni manuali**
 - le tecniche più **efficienti** per la individuazione di **errori/difetti**
 - partecipano alle riunioni di revisione **l'autore dell'SRS, il cliente, un progettista, un esperto di qualità**
 - ogni partecipante rivede l'SRS prima della riunione
 - uso di **checklist** durante l'ispezione
- **Tecnica di Reading**: qualcuno diverso dall'autore legge l'SRS per coglierne potenziali problemi (ambiguità, ...)
- Uso di Scenari relativi alle varie modalità operative del sistema

TOPIC 3

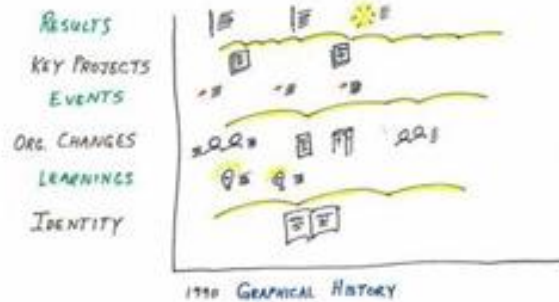
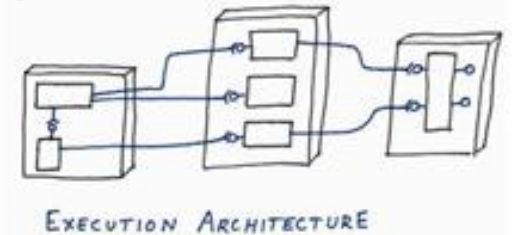
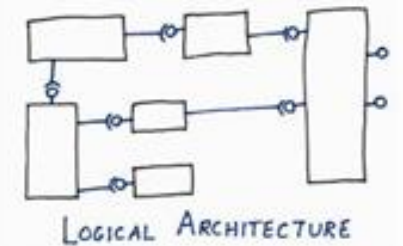
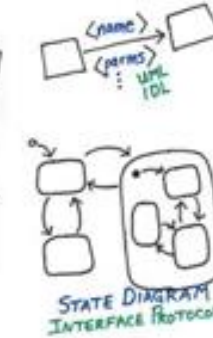
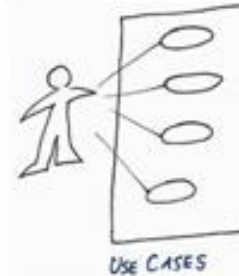
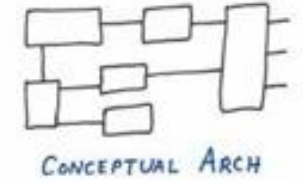
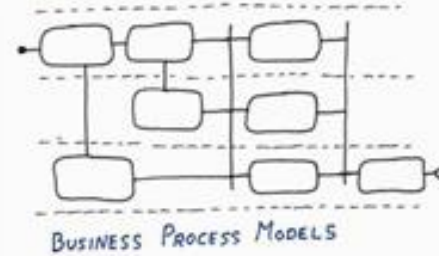
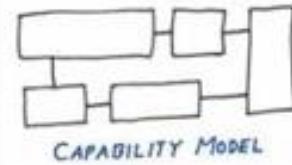
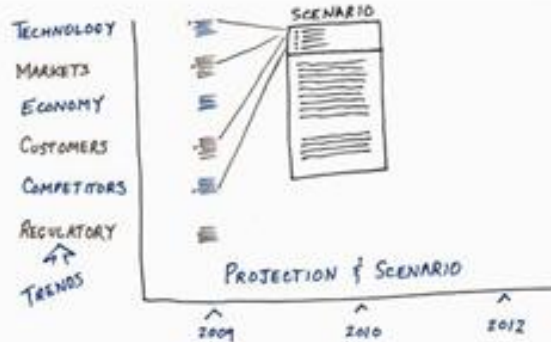
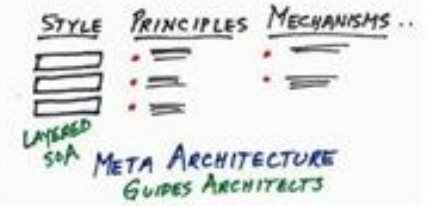
Classificazioni Requisiti

- Se chiedete ad un ingegnere di **mostrare la struttura di un edificio** e documentarla egli vi presenterà planimetrie del sito, dei piani della struttura, viste di elevazione, sezioni, disegni dettagliati, ecc. ...

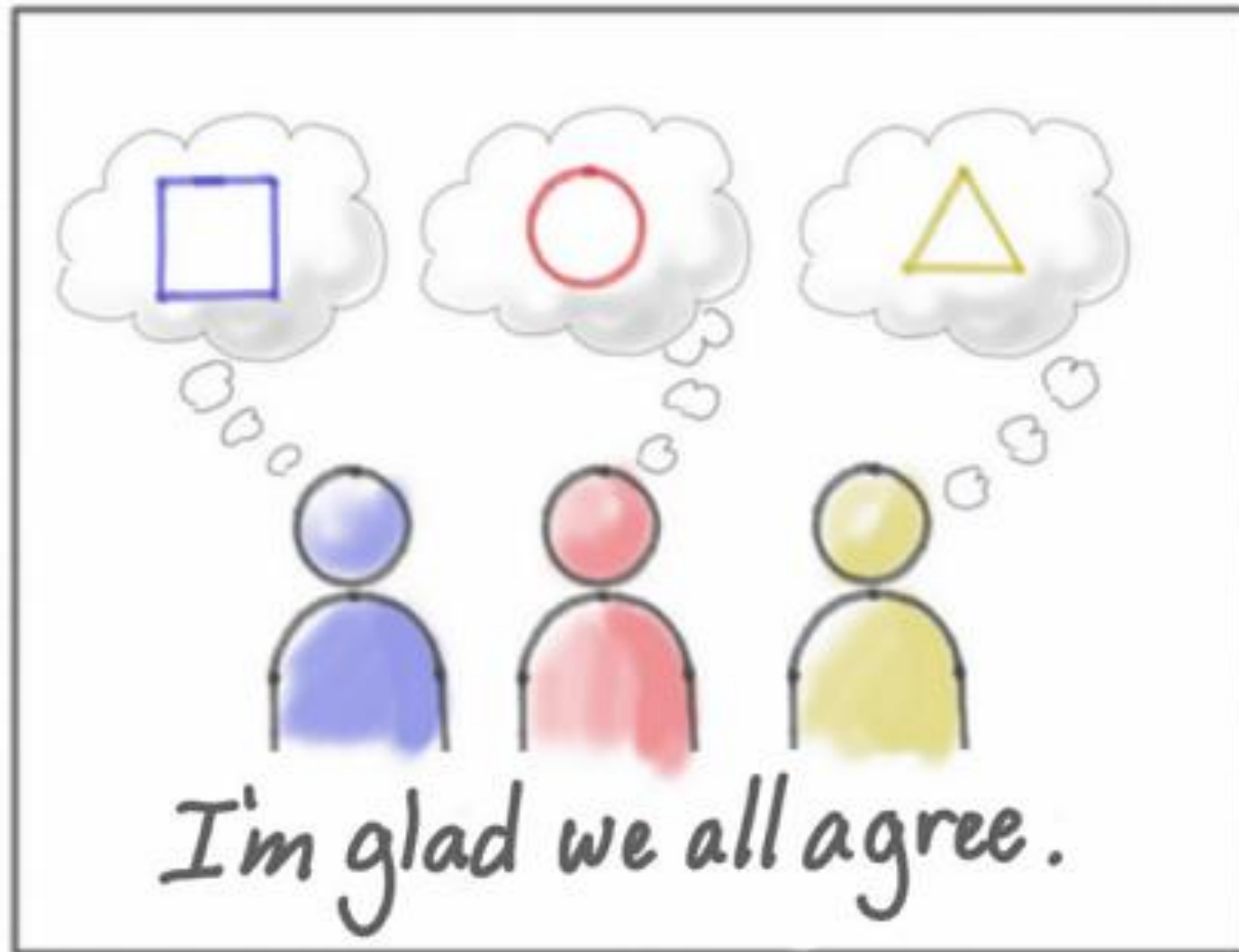


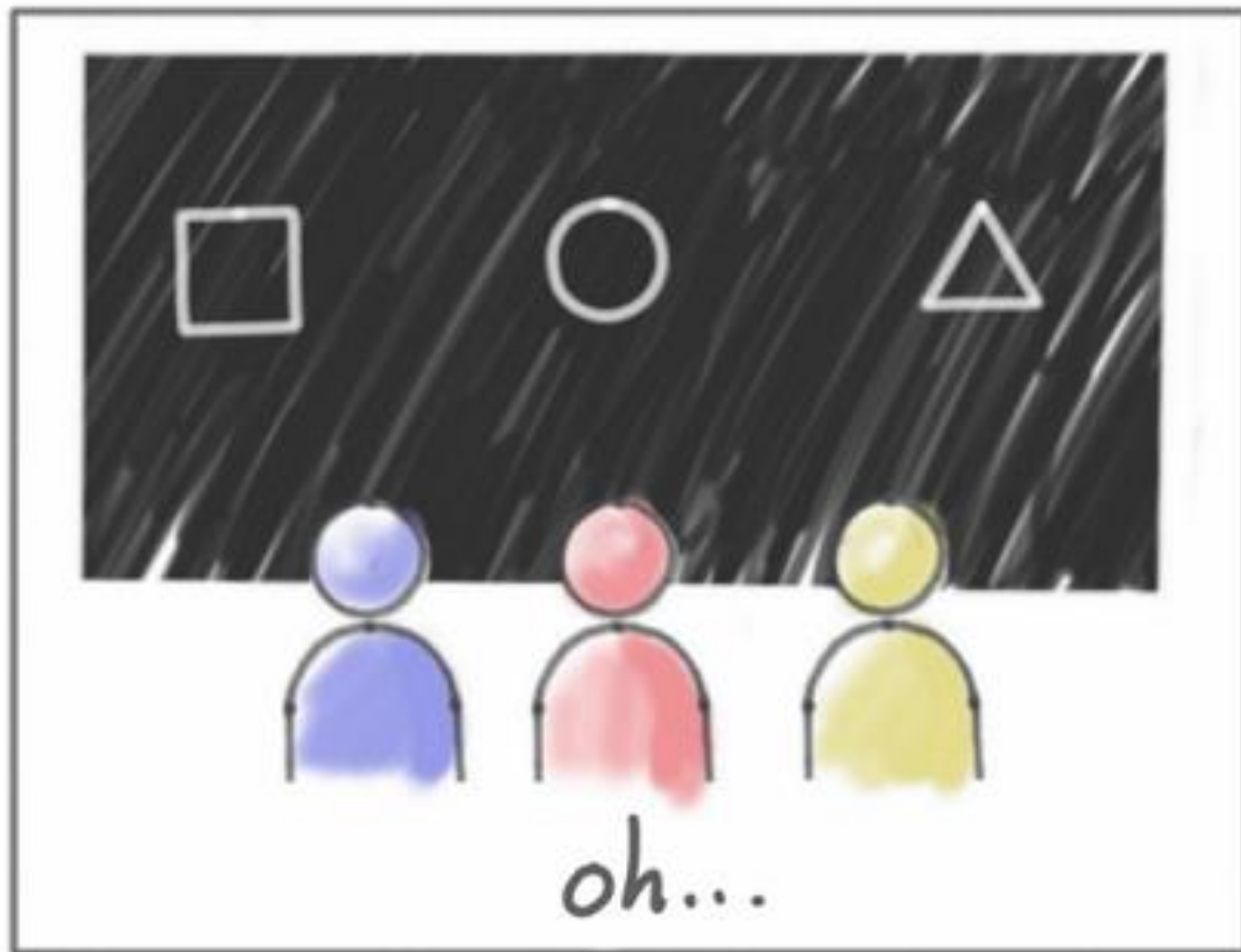
- Se chiedete invece a un ingegnere del software di **mostrarvi e documentarvi l'architettura di un sistema con dei diagrammi** ?
- Otterrete un insieme vario di «**scatole e linee**» di diverso tipo e dettaglio
- Probabilmente se lo chiederete a mille persone diverse otterrete mille schemi e documenti differenti.

The Visual Architecting Process



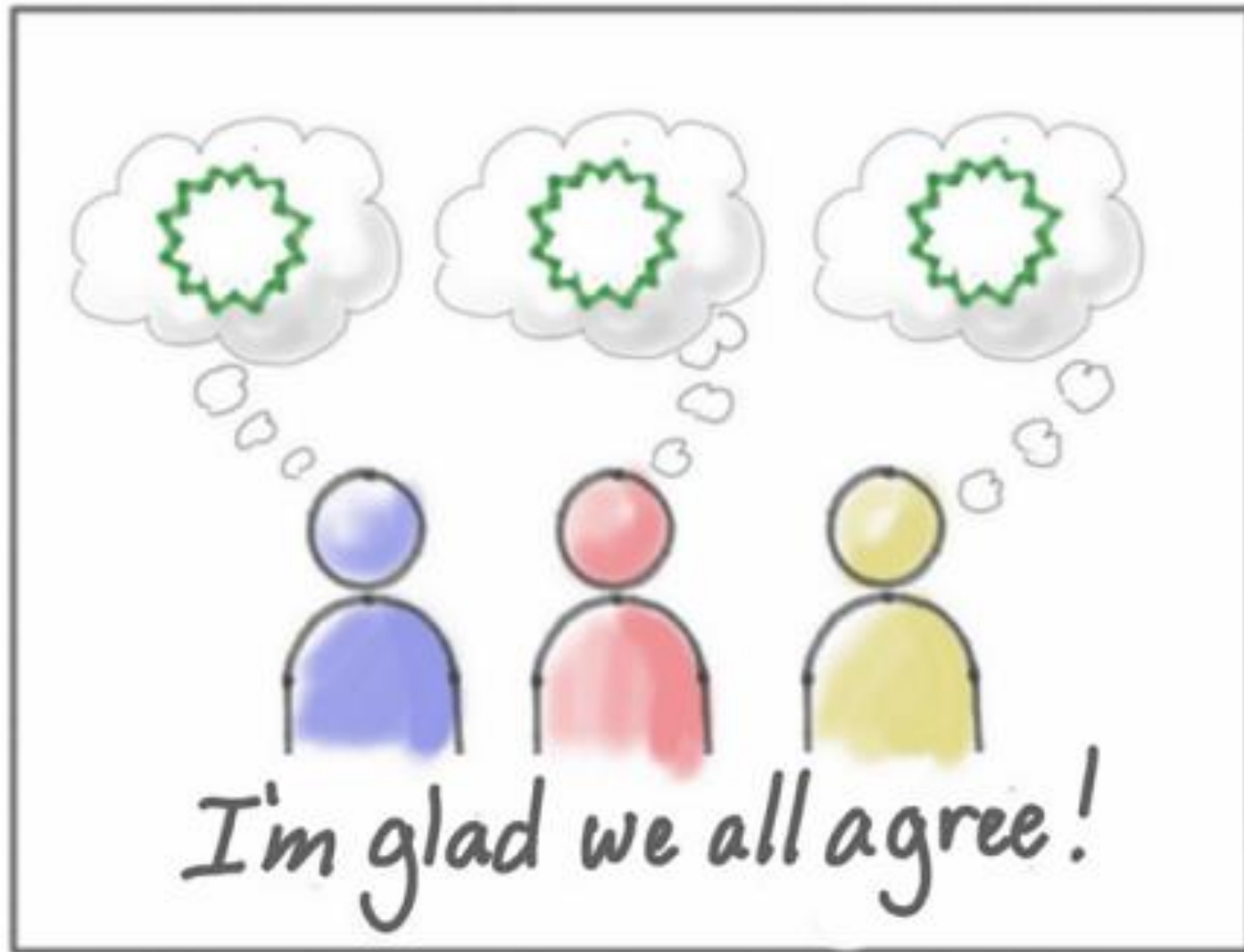
«Sono contento che siamo tutti d'accordo»



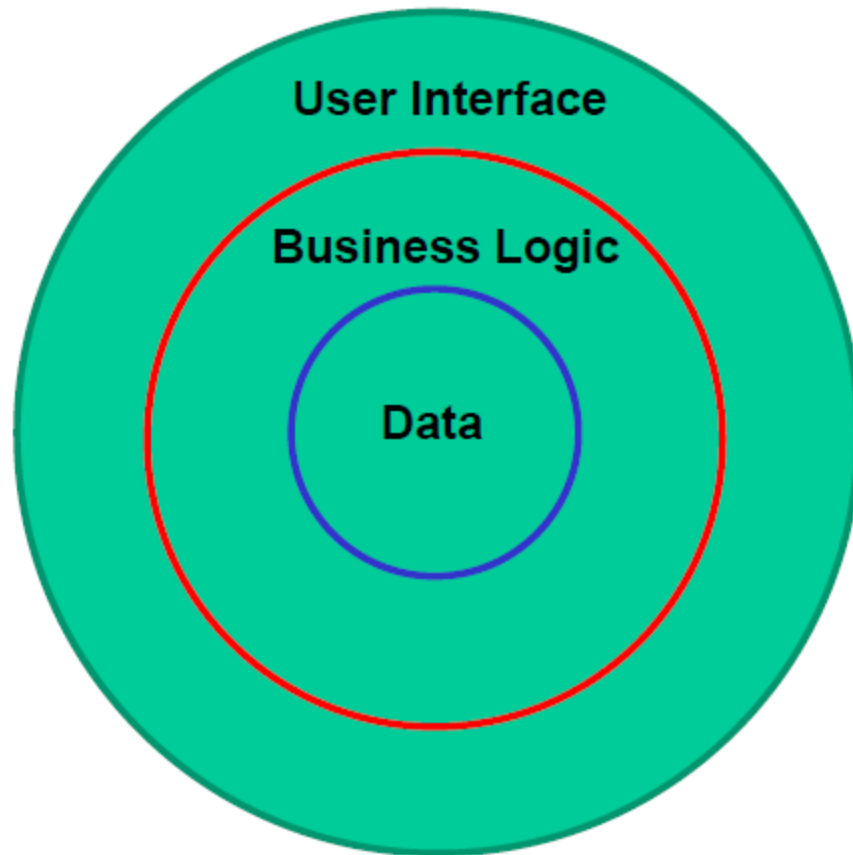




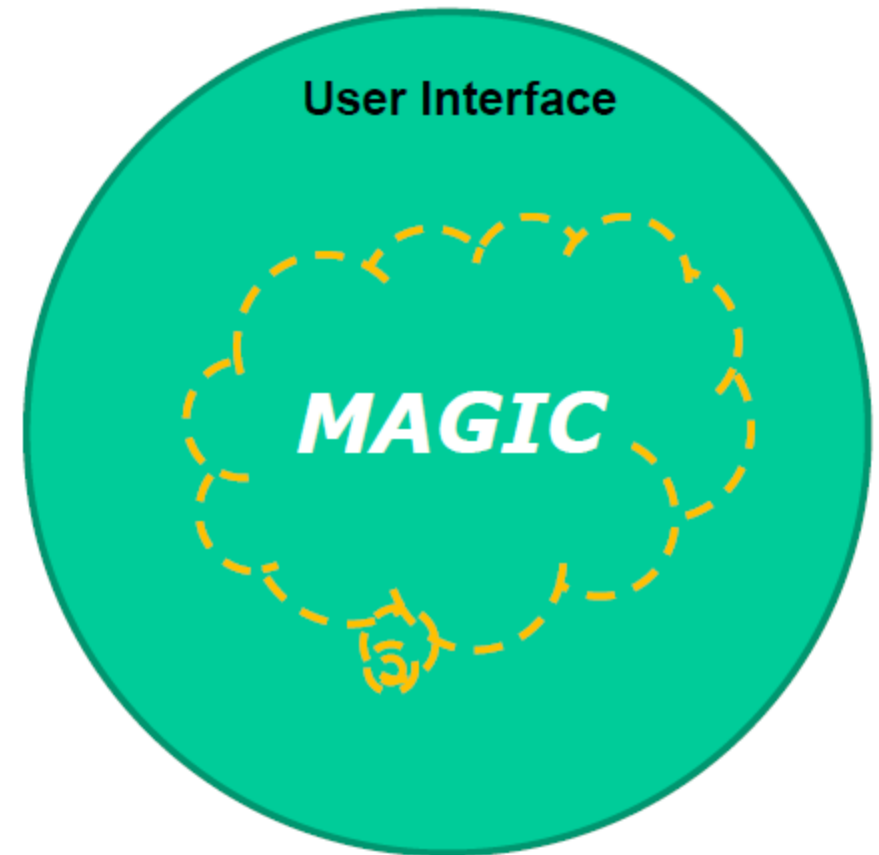
«Sono contento che siamo tutti d'accordo»



Il vostro software



Come lo vede l'utente



- **Fase preliminare:** individuazione degli stakeholder e impostazione della raccolta
- **Raccolta** (elicitation): fase in cui si raccolgono i requisiti dalle persone coinvolte, usando tecniche precise
- **Analisi dei requisiti:**
 - organizzazione dei requisiti dal punto di vista degli sviluppatori: definizione di un modello del sistema
 - eliminazione di contraddizioni e ambiguità
 - negoziatura su che cosa effettivamente realizzare, in che tempi e con quali costi

- **Requisiti funzionali:**

- Sono elenchi di servizi che il sistema deve fornire (in alcuni casi possono affermare esplicitamente cosa il sistema non dovrebbe fare)
- Servizi forniti, reazione a input specifici , comportamenti in circostanze specifiche

- **Requisiti non funzionali**

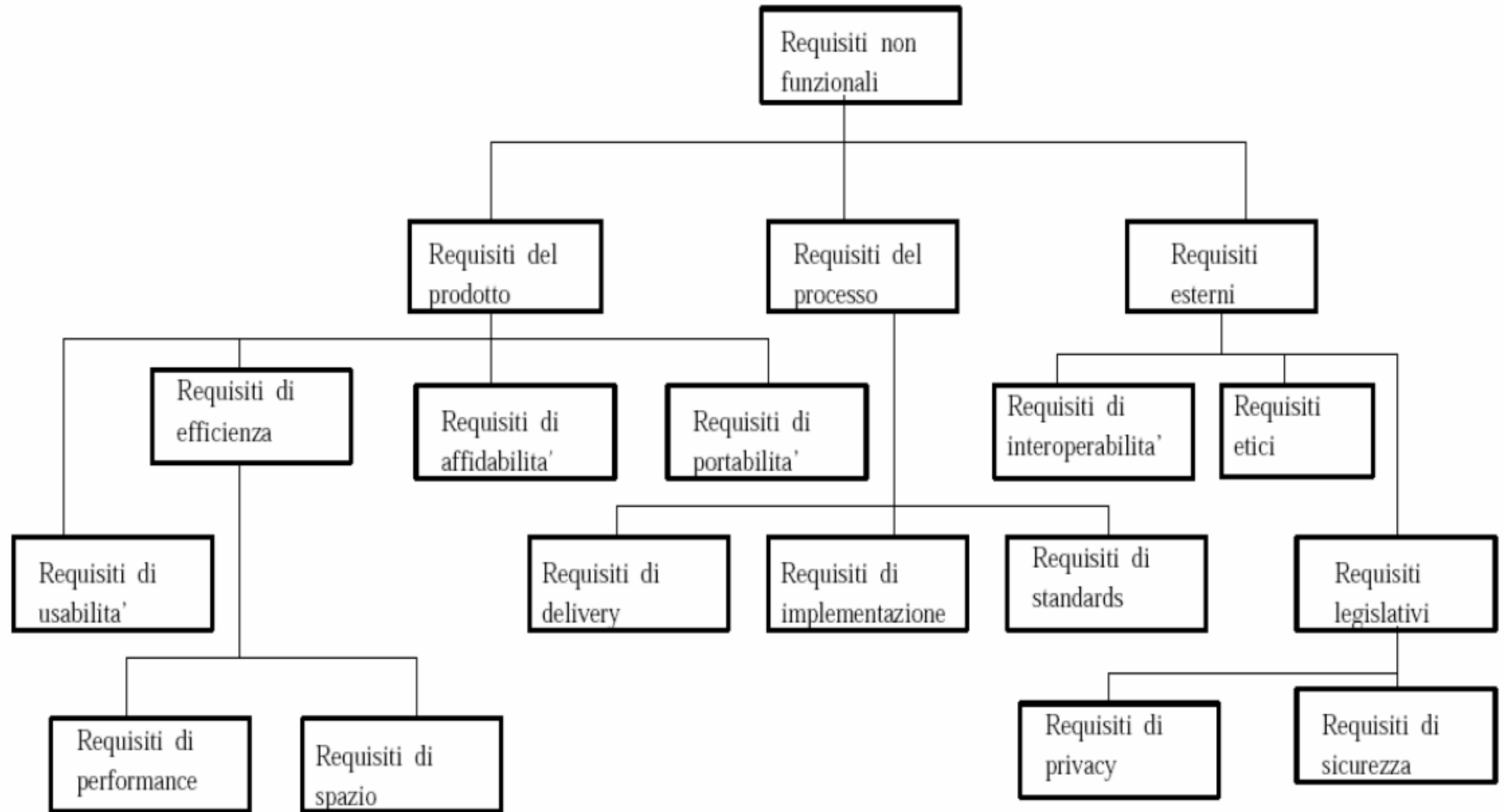
- Sono vincoli sui servizi offerti dal sistema (es. vincoli temporali, sul processo di sviluppo, sugli standard utilizzati, ecc.)
- Proprietà user-visible relative al sistema nel suo complesso
- Sicurezza, privacy, usabilità, affidabilità, disponibilità, prestazioni, ...

- **Requisiti di dominio:**
 - Derivano dal dominio di applicazione del sistema, ne riflettono le caratteristiche e possono essere requisiti **funzionali o non funzionali**.
- **Requisiti informativi:**
 - Identificano le principali informazioni di business che il sistema deve gestire, la loro struttura e le loro eventuali relazioni (i processi di business sono identificati dai requisiti funzionali)
- **Vincoli**
 - Imposti dal cliente. Non hanno un effetto diretto sul sistema dal punto di vista dell'utente (es: linguaggio di programmazione, piattaforma di sviluppo)

Classificazione dei requisiti non funzionali

- **Requisiti del prodotto:** Requisiti che specificano che il prodotto deve comportarsi in un certo modo, per esempio velocità di esecuzione, affidabilità, ecc.
- **Requisiti organizzativi o del processo:** Requisiti che sono conseguenza di politiche aziendali e procedure, per esempio il processo utilizzato, mezzi di implementazione, ecc.
- **Requisiti esterni:** Requisiti che sorgono da fattori che sono esterni al sistema ed al suo processo di sviluppo, per esempio requisiti di interoperabilità, requisiti legislativi, ecc.

Tipi di requisiti non funzionali



Misure dei requisiti non funzionali

Proprietà	Misura (Esempi)
Velocità	Transazioni al secondo processate Tempo di risposta utente/evento Tempo di Refresh dello screen
Dimensione	Kbytes Nr. Chips di Ram
Facilità D'Uso	Tempo di Training Nr. Di Frames di Help
Affidabilità	Mean Time to Failure Probabilità di non disponibilità Frequenza di occorrenza dei fallimenti Disponibilità
Robustezza	Tempo di restart dopo il crash Percentuale di eventi che causano il crash Probabilità che I dati siano corrotti dopo un crash
Portabilità	Percentuale di codice che dipende dal target Nr. di sistemi Target

- Derivano dal **dominio di applicazione del sistema** piuttosto che dalle specifiche necessità degli utenti
- Possono essere nuovi requisiti funzionali, porre nuovi vincoli o definire particolari calcoli
- Se questi requisiti non sono soddisfatti, potrebbe essere **impossibile** far sì che il sistema lavori in modo soddisfacente

- Solitamente molti sistemi software operano in un ambiente che include altri sistemi per cui potrebbero doversi **interfacciare con questi ultimi in diversi modi** (tali specifiche dovrebbero essere definite in fase di analisi dei requisiti ed incluse nel documento dei requisiti)
- Tre tipi di interfacce potrebbero essere definite in un documento di specifica dei requisiti:
 - **Interfacce procedurali**: servizi offerti da sottosistemi già esistenti
 - **Interfacce dei dati**: strutture dati che vengono trasmesse da un sottosistema all'altro
 - **Interfacce di rappresentazione**: specifici pattern utilizzati per descrivere dati

TOPIC 4

Osservazioni sui Requisiti

- I **confini** del sistema sono mal definiti.
- Sono fatte assunzioni di progetto **non necessarie**.
- Gli «**Stakeholder**» non conoscono compiutamente le proprie necessità
- Gli «**Stakeholder**» hanno una scarsa conoscenza delle possibilità e delle limitazioni del computer
- Gli «**Sviluppatori**» non hanno una buona conoscenza del dominio

- Gli «Stakeholder» e gli «Sviluppatori» parlano linguaggi differenti
- Si **omette** di riportare dell'informazione «**ovvia**».
- «Stakeholder» diversi hanno diverse viste del sistema che risultano in **conflitto** tra loro
- I requisiti sono **vaghi e non testabili**, esempio il sistema deve essere «**user friendly**», ma non si specifica che cosa ciò vuole dire
- I requisiti sono volatili e cambiano nel tempo.

- **Comprensibili**
 - Eliminare confusione e incomprensioni
 - Linguaggio e termini specifici del dominio confondono gli sviluppatori
 - I termini tecnici confondono gli stakeholder
 - *Usare* frasi brevi e dichiarative
 - Esempi, figure e tabelle possono aiutare ...
 - ... oppure discuterli in dettaglio col cliente **durante lo sviluppo**

- **Non prescrittivi:** Scrivere cosa vuole il cliente e **non cosa vorrebbe fare lo sviluppatore**
- **Concisi**
 - Facilitano la validazione da parte del cliente
 - Sono più facilmente gestibili
- **Corretti e Completi:** Lista esaustiva di requisiti, oppure facilità a correggerli e integrarli anche durante lo sviluppo

- **Linguaggio Consistente**
 - Nessuna contraddizione
 - Dovere (shall) implica un obbligo
 - Dovrebbe (should) implica un'opzione desiderabile
- **Tracciabili**
 - I requisiti devono essere **identificati in modo univoco**
 - Corrispondenza tra le varie parti di analisi, progetto, codice ed i requisiti rilevanti per tale parte

- **Non ambigui → testabili**
 - Scrivere test case (test di accettazione) durante la raccolta dei requisiti
 - Coinvolgere il cliente il prima possibile
 - Dare una definizione quantitativa per ogni avverbio e aggettivo
 - Rimpiazzare i pronomi con i nomi specifici delle entità
 - Ogni nome deve essere definito in modo esatto nel documento dei requisiti
 - Oppure: si possono discutere in dettaglio col cliente durante lo sviluppo

- **Ordinati per importanza e stabilità**
 - Le priorità dovrebbero essere decise insieme al cliente
 - E' necessario un processo di negoziazione per stabilire:
 - Priorità reali
 - Quanto un requisito è volatile
- **Fattibilità**
 - I **requisiti non fattibili** individuati durante la fase di raccolta dei requisiti **devono essere chiariti subito**
 - I **requisiti non fattibili** individuati durante la fase di analisi **devono essere notificati** e richiedono un aggiornamento del documento dei requisiti

- Vi sono vari modi per esprimere i requisiti:
 - Come Documento dei Requisiti
 - Come Casi d'Uso
 - Come User Stories
 - Con metodi e linguaggi di specifica formale

Classica:

- l'applicazione gestirà i prestiti dei volumi agli iscritti

User story:

- Come iscritto voglio consultare il catalogo per prendere in prestito un volume

Caso D'Uso:

- Un iscritto interroga la funzione Catalogo per cercare un volume, il Catalogo chiede le informazioni di ricerca,...

- **Requisiti:** attributi, capacità, caratteristiche, standard seguiti, qualità del sistema
- **Analisi e Progettazione:** relazione tra sw ed esterno, tra le componenti interne, principi sw usati per i componenti, Software Design Specificaion
- **Documentazione tecnica:** Software Detailed Design Specification, documentazione API, algoritmi
- **Documentazione di test:** scrittura test case, test automatici e manuali, performance test
- **Documentazione utente:** user manual, help on line, corsi, release notes
- **Documentazione di qualità:** se adottiamo standard
- **Marketing:** pubblicizzazione prodotto, analisi di mercato

TOPIC 5

Approccio «Classico»

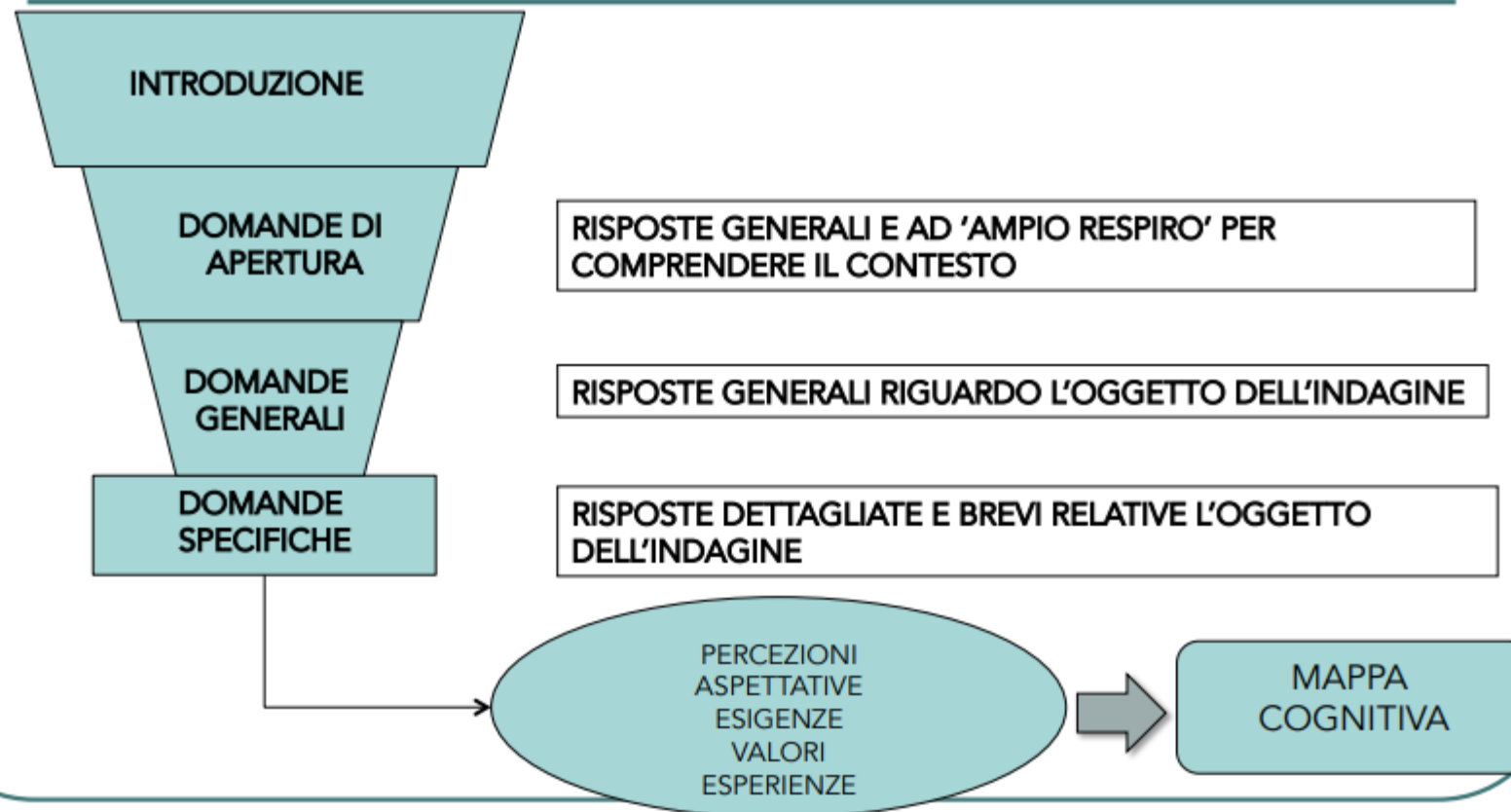
- L'**elicitazione dei requisiti** (requirement elicitation) è un insieme di diverse tecniche, combinate fra loro, per riuscire ad «estrarre dalla mente del cliente o committente le idee per capire cosa il software oggetto del contratto dovrà fare».
- Le tecniche di **elicitazione** più usate sono:
 - le **interviste**, durante le quali si pongono alle persone del cliente domande esplicite;
 - l'**osservazione**, durante la quale si osservano «sul campo» le funzioni svolte in cui il software dovrà intervenire;
 - il gruppo di lavoro (brainstorming).
 - ...

- **Approccio a Imbuto:** è di tipo **deduttivo** ove l'intervistatore parte da domande molto generali per poi restringere l'argomento dell'intervista a temi specifici
- Questo approccio è utile nel caso in cui l'intervistato sia emozionato o eccessivamente deferente, poiché il fatto che le domande di carattere generale (normalmente in forma aperta) non prevedano una risposta «sbagliata» allevia la tensione dell'intervistato.
- Si parte da uno schema generale per arrivare ad uno specifico, da quelle meno strutturate a quelle più strutturate.
- Le domande devono risultare spontanee e semplici e **non devono suggerire** alcuna potenziale risposta

- In generale possiamo avere **cinque tipologie di domande** (dipende dal conducente e dalla percezione iniziale che ha dell'intervistato):
 - **domande di apertura**: permettono la creazione del gruppo
 - **domande di introduzione**: portano i partecipanti a riflettere sull'oggetto della discussione
 - **domande di transizione**: conducono alla chiave del tema di studio
 - **domande chiave**: rappresentano il cuore del tema trattato e per questo motivo richiedono maggiore attenzione da parte del moderatore
 - **domande finali**: portano alla chiusura della discussione e permettono ai partecipanti di riflettere sui precedenti commenti.

Traccia di intervista

LOGICA A IMBUTO



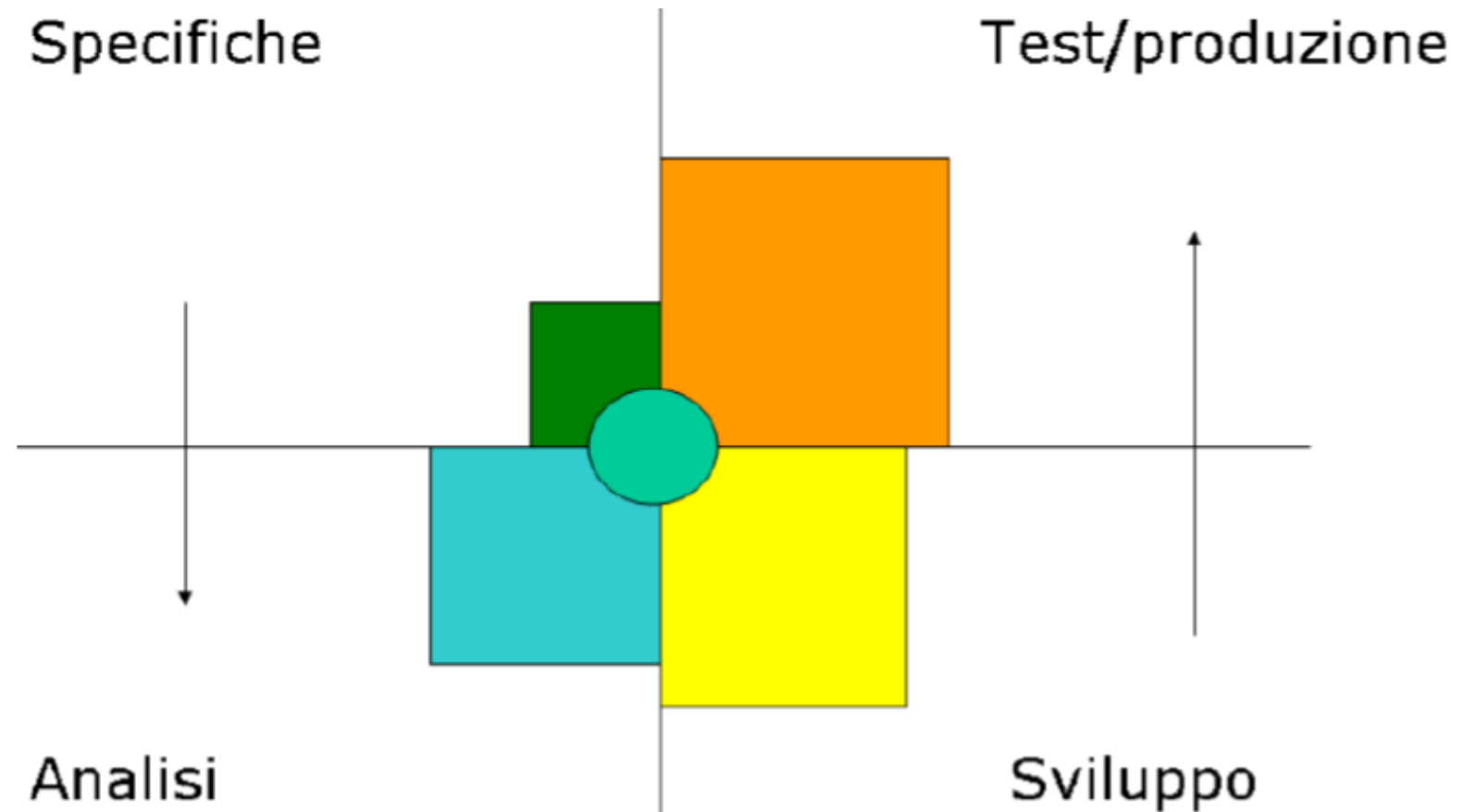
- **Approccio a Piramide:** E' un approccio induttivo
- L'**intervistatore** parte da domande molto dettagliate per poi ampliare l'argomento dell'intervista mediante domande aperte che richiedono risposte più generali.
- Questo tipo di intervista permette di superare la riluttanza di un intervistato scettico poiché inizialmente non richiede un forte coinvolgimento da parte dell'intervistato.

Le interviste A Piramide Vs A Imbuto



- Serve per descrivere in modo formale e preciso i requisiti che dovrà avere il sistema oggetto del progetto.

Propagazione
errori





Come viene spiegato dal cliente



Come viene compreso dal Capo Progetto



Come viene analizzato dall' analista



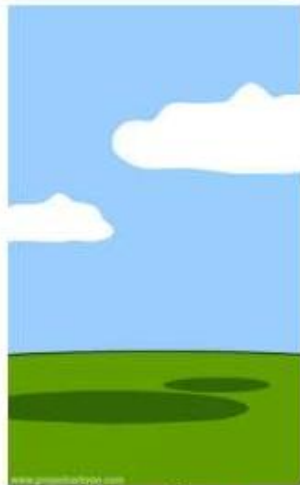
Come viene scritto dal Programmatore



Quello che riceve il Beta Tester



Come viene descritto dal Venditore



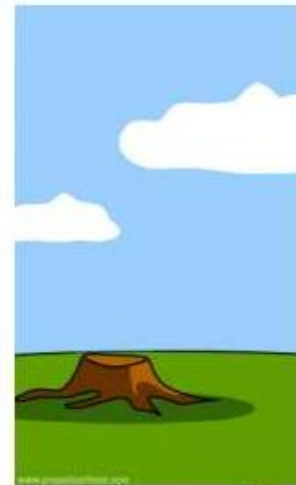
Come viene documentato il progetto



Quali sono le funzioni installate



Come è stato fatturato al cliente



Come viene supportato



Quale pubblicità viene fatta



Quello di cui ha veramente bisogno il cliente

- Produrre una **descrizione completa formalizzata lato utente**, con il giusto livello di dettaglio, di :
 - Tutto ciò che il sistema deve fare (requisiti funzionali),
 - Dell'ambiente in cui dovrà operare (requisiti non funzionali)
 - Dei vincoli che dovrà rispettare.
- Formalizzazione dei **documenti**:
 - Usando anche diagrammi **UML** ...
 - ... o **Agile Methodologies**

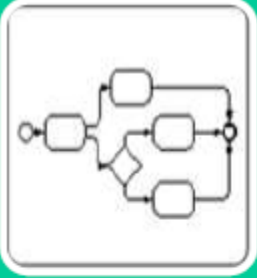


- La fase di **progettazione** deve produrre le istruzioni operative per la realizzazione effettiva del progetto informatico (implementazione).
- Le **istruzioni** devono avere il giusto livello di dettaglio ed essere espresse in forma di documenti strutturati.
- Risultato: **la definizione dell'architettura del sistema**
 - i suoi componenti software
 - le proprietà visibili esternamente di ciascuno di essi (l'interfaccia dei componenti)
 - le relazioni fra le parti.
- Per la **progettazione** si possono usare diagrammi gli **UML Activity Diagram** (o **BPMN ...**)

Documentazione tecnica e cambiamenti nel team



- **Test**
 - Report Unit Testing o Automated Tests
 - Report Test Manuali eseguiti/falliti
 - Report Test di performance
- **Delivery**
 - Documentazione on line o inclusa nel setup (meglio «how to» piuttosto che spiegazione testuale)
 - Release notes
 - Corsi (on line o in aula)



Raccolta requisiti

- BPMN
- Testuale

```
if(!searchString){  
    return [];  
}
```

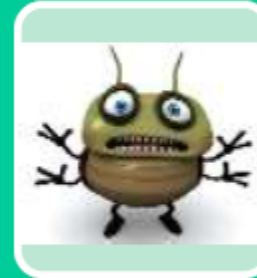
Implementazione

- Software
- Code documentation



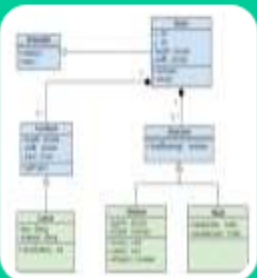
Analisi

- UML Use Case Diagram
- UML Activity Diagram



Test

- Test plan
- Unit Testing / TDD
- Performance Test



Progettazione

- UML Class Diagram
- UML Sequence Diagram



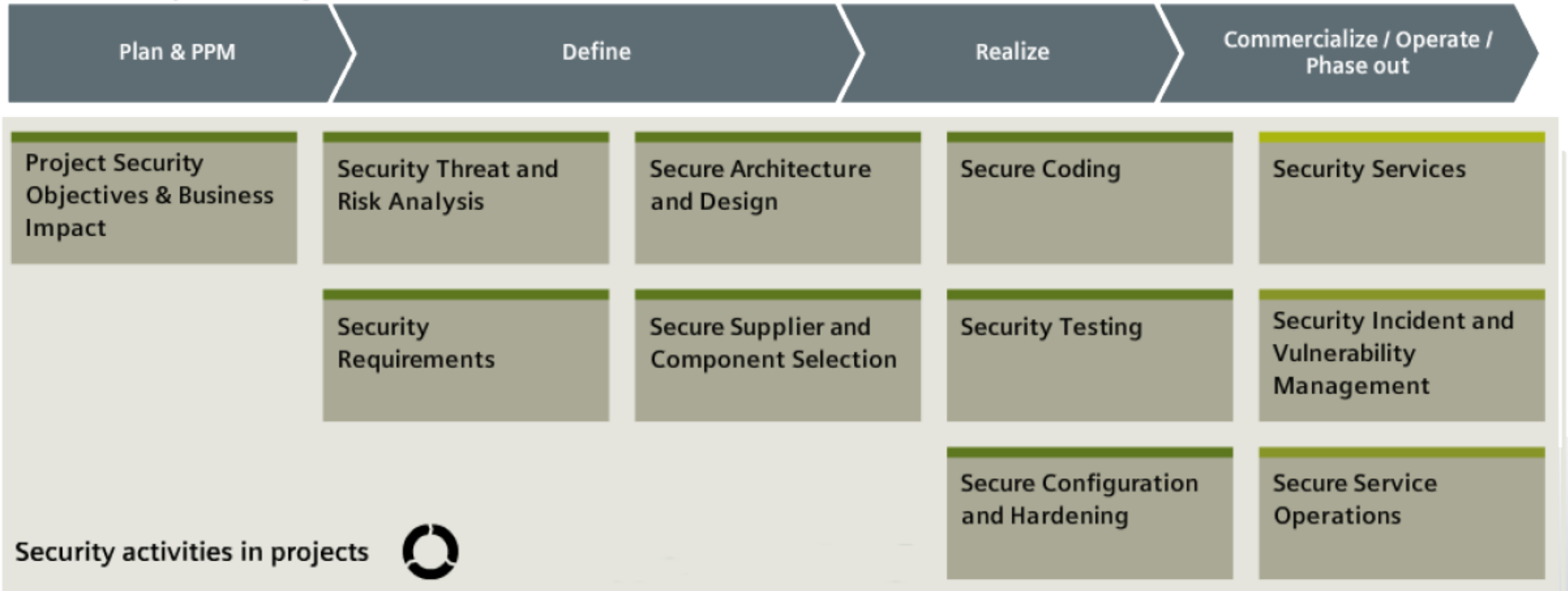
Rilascio

- User manual
- Technical reference manual

- ISO/IEC 9001: Sistema di gestione per la qualità
- ISO/IEC 90003: Applicazione della ISO/IEC 9001 al software
- ISO/IEC 29110: Software Lifecycle Processes for VSE
- ISO/IEC 29119: Software Testing
- ISO/IEC/IEEE 29148-2011 : Systems and software engineering - Life cycle processes - Requirements engineering
- ISO/IEC 9126: Modello della qualità del software
- ISO/IEC 12207: Software Lifecycle Processes
- ISO/IEC 15288: LCM - Lifecycle Processes
- ISO/IEC 15289: Lifecycle Product Information Content (documentation)
- ISO/IEC 15504: Software Process Assessment
- ISO/IEC 16326: Project Management
- ISO/IEC 19011: Guida alla conduzione degli audit dei sistemi di gestione
- ISO/IEC 19759: Software Engineering Body of Knowledge (SWEBOK)
- ISO/IEC 27001 e 27002: Information technology – Security techniques –Information security management systems – Requirements

- ISA/IEC-62443 (ISA-99) serie di standard che definisce procedure per implementare in modo elettronicamente sicuro gli Industrial Automation and Control Systems (IACS)

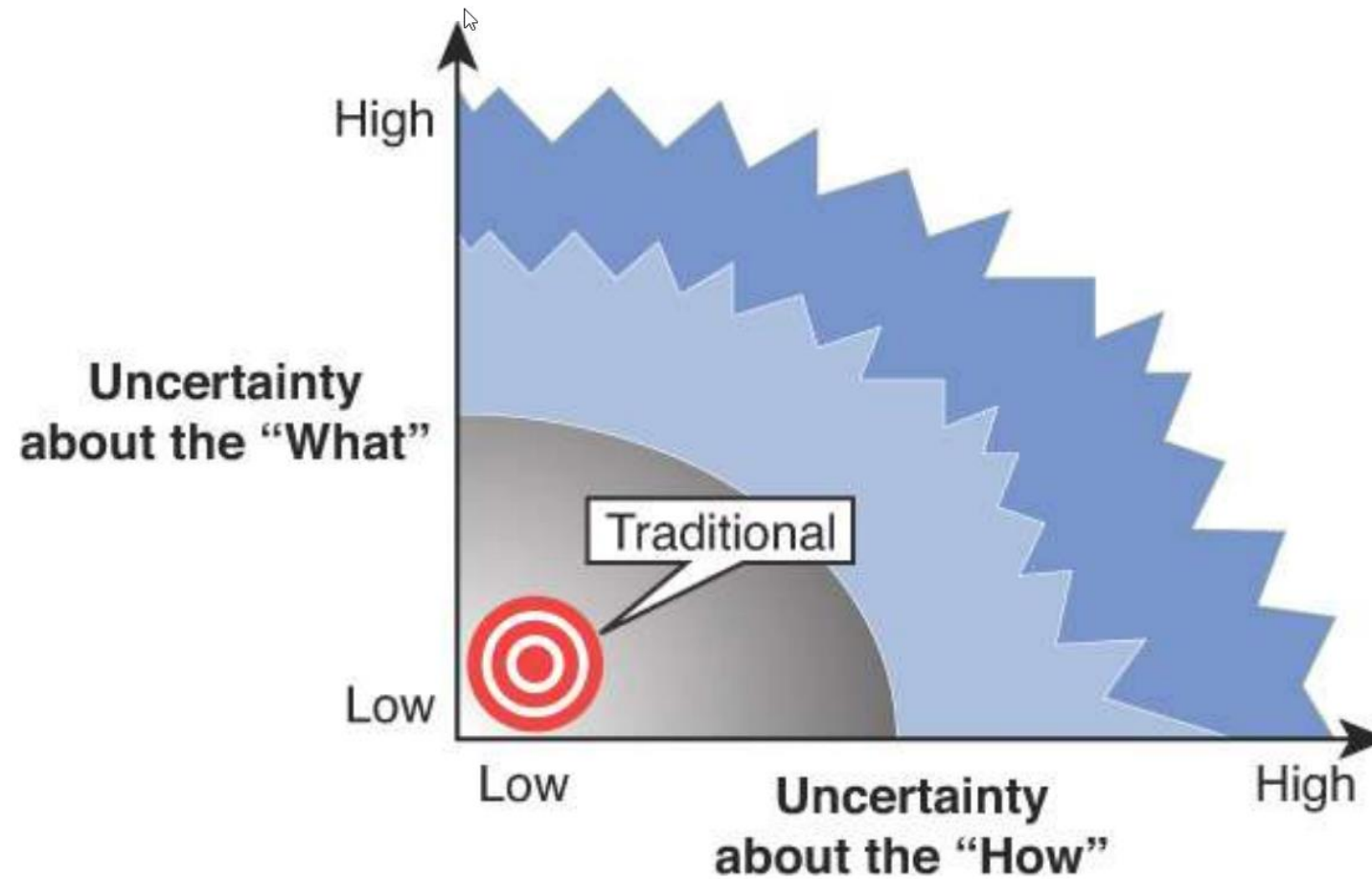
Product Lifecycle Management



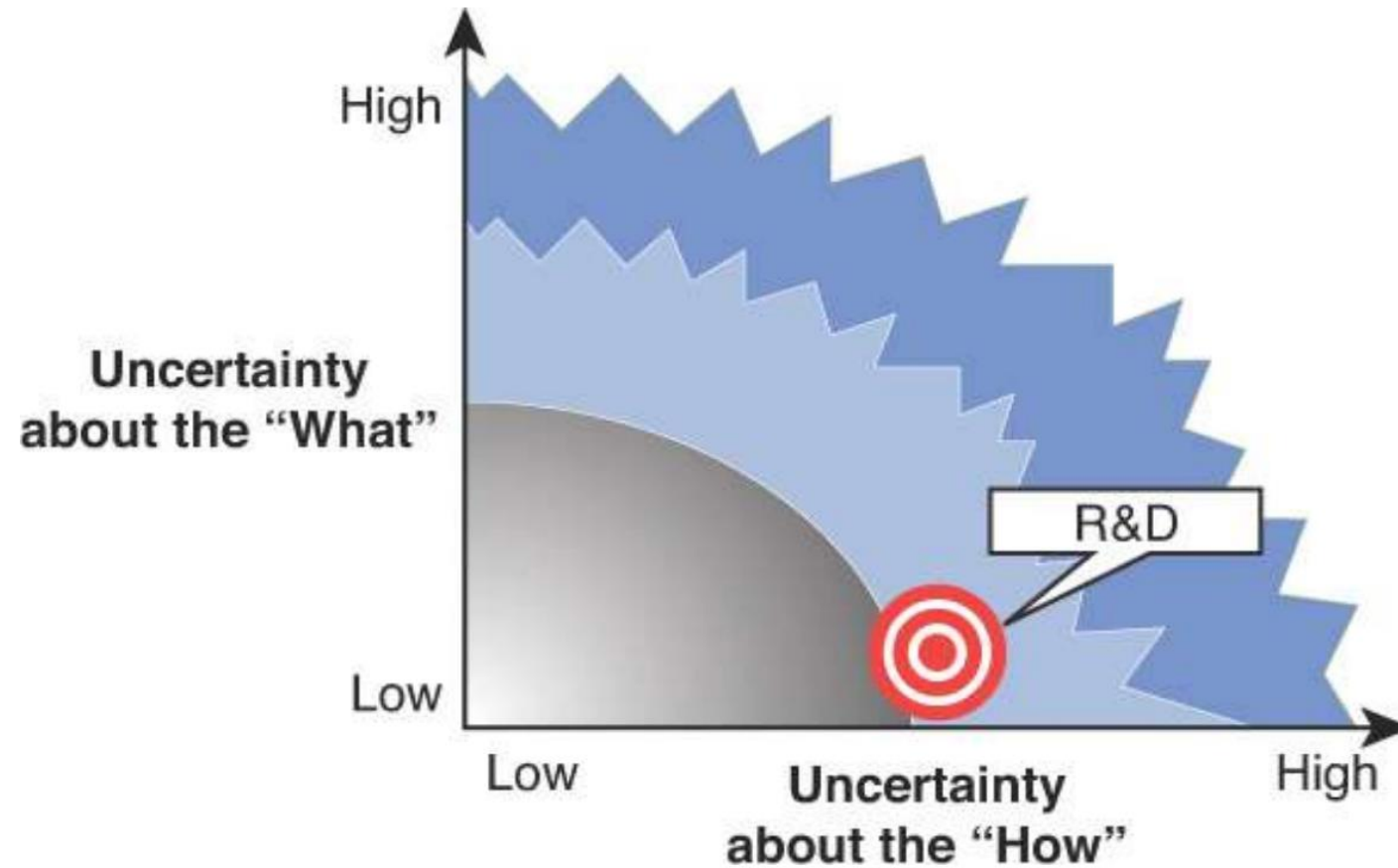
TOPIC 6

Test

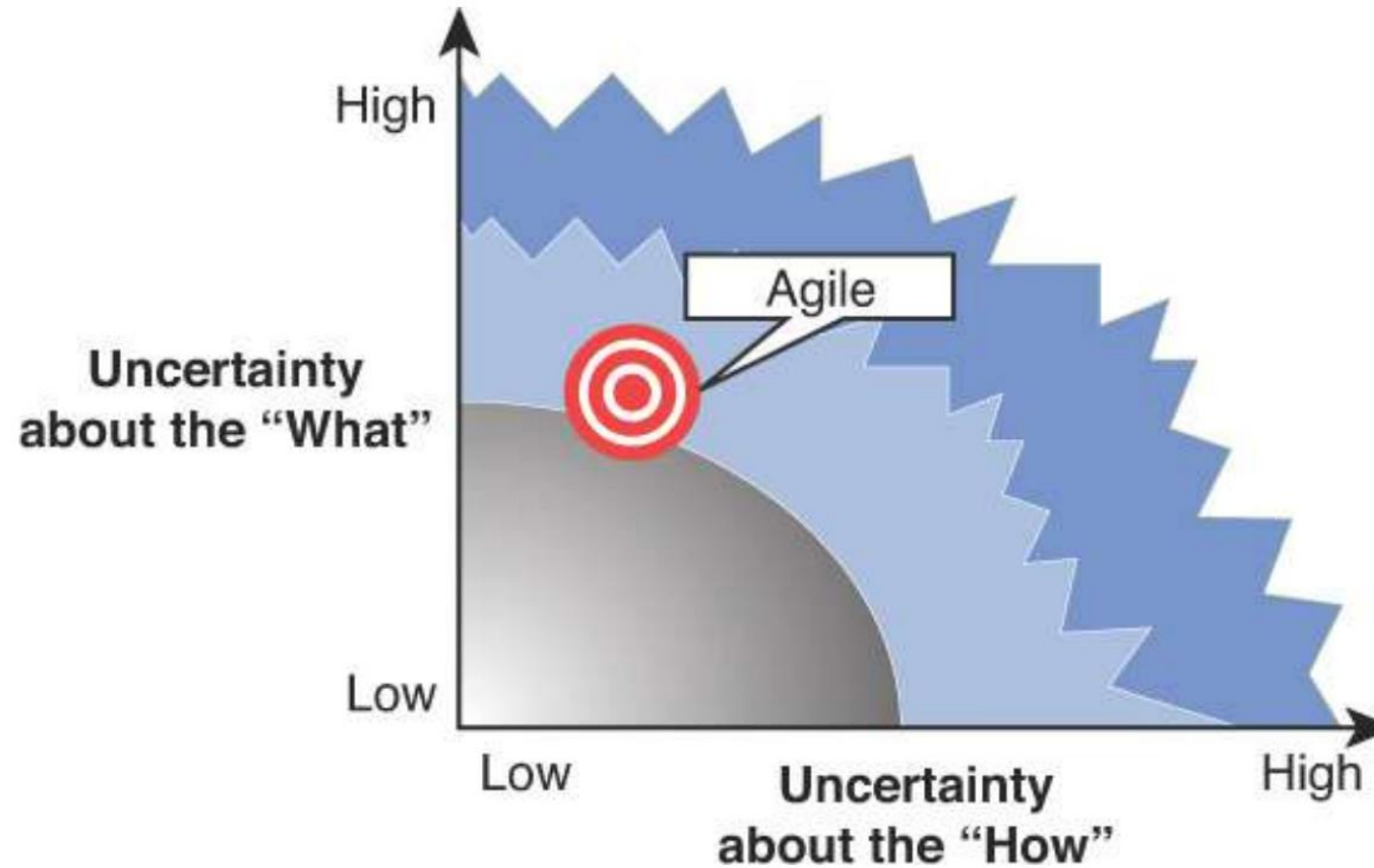
- Quando l'incertezza è bassa e la conoscenza del dominio alta, l'enfasi è sulla conformità.



- Quando gli obiettivi sono chiari, ma non esiste una soluzione nota, l'enfasi è sulla fattibilità.



- Quando gli obiettivi sono mutevoli o incerti, l'enfasi è sull'apprendimento



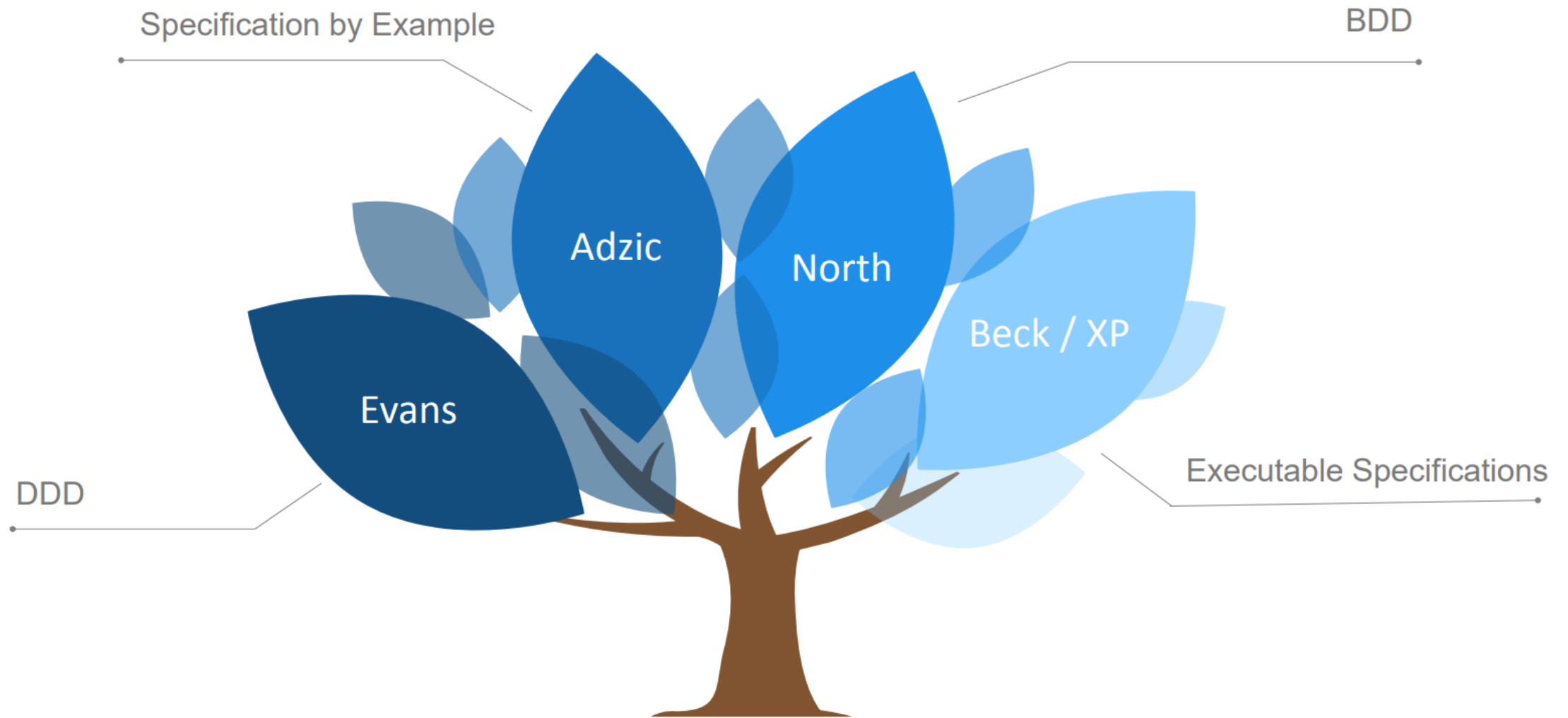
Ok, I want to do the thing right, but how I know it is the right thing?

(Ok, voglio fare la cosa giusta, ma come so che è la cosa giusta?)

Cosa sono gli AT (Acceptant Test)

- Gli AT non sono test di accettazione tradizionali: Non sono eseguiti dall'utente finale dopo l'implementazione, al fine di verificare se il sistema funziona secondo le specifiche contrattualizzate
- Gli AT non sono System Tests: Non sono test scritti dai tester leggendo il documento dei requisiti
- Gli AT non sono Acceptance Criteria: Non sono informali
- Gli AT sono indipendenti dall'implementazione: Validano innanzitutto i requisiti, e poi possono essere usati per testare l'implementazione
- Gli AT sono definiti collaborativamente: Nascono dalla stretta collaborazione tra cliente/utente, sviluppatore e tester e sono il pilastro su cui fondare una comune comprensione del dominio dell'applicazione

L'Albero della conoscenza (dei requisiti) del team.



- **ATDD** – Acceptance Test Driven Development: Il focus è sul concetto di “accettazione” e su quello di verifica. E’ naturale il nesso con i cosiddetti Acceptance Criteria, che sono parte essenziale della DoR (Definition of Ready) e della DoD (Definition of Done)
- **BDD** – Behaviour Driven Development (North): L’attenzione è sul comportamento del sistema. È forse l’acronimo più popolare.
- **Specification by Example** (Adzic): La definizione di Adzic si libera del “DD” e mette al centro il processo di costruzione delle specifiche tramite esempi
- **Executable Specifications**: Qui l’accento è sulla “specifica” come vero e proprio test eseguibile e automatico
- **DDD** – Domain Driven Design: Ubiquitous language! Conoscere il dominio per avere il dominio

Test-driven development (TDD)

- E' una tecnica di **automazione** degli «**unit test**» utilizzata per guidare lo sviluppo software e forzare (nonché favorire) il disaccoppiamento delle dipendenze.
- Il risultato di questa pratica è **una serie completa di test di unità** che possono essere eseguiti in qualsiasi momento per fornire un feedback sullo stato di funzionamento del software mentre lo si sta ancora sviluppando.
- Il concetto è «far funzionare qualcosa adesso e perfezionarlo in seguito».
- Dopo ogni test viene eseguito il **refactoring**; successivamente viene rieseguito lo stesso test o un test simile.
- Il processo viene ripetuto tutte le volte necessarie fino a quando ogni unità non funzioni secondo le specifiche desiderate.

Test-driven development (TDD)

- Riepilogando, tale pratica prevede l'utilizzo di cicli brevi di sviluppo e verifica, in cui dapprima si scrive un **test automatico** per le funzionalità da sviluppare, il quale, ovviamente, **fallisce**.
- Successivamente si scrive la **minima quantità di codice** che consente il superamento del test;
- Infine si passa al **refactoring** ossia alla ristrutturazione del codice sulla base dei risultati ottenuti.
- Con questa pratica, si riescono a individuare con esattezza specifiche e caratteristiche del codice, perché ne viene verificato, a brevi intervalli, il comportamento «reale», che imita le situazioni a cui sarà effettivamente sottoposto.
- Si ottiene perciò codice funzionante, affidabile e generalmente più pulito.

Acceptance Test Driven Development (ATDD)

- È una **tecnica** utilizzata per coinvolgere i clienti nel processo di progettazione del test prima dell'inizio della codifica.
- È una **pratica collaborativa** in cui utenti, tester e sviluppatori definiscono criteri di accettazione **automatizzati**.
- ATDD aiuta a garantire che tutti i membri del progetto **comprendano esattamente** cosa deve essere fatto e implementato
- I test non riusciti forniscono un rapido feedback in merito al mancato soddisfacimento dei requisiti.
- I test sono specificati in termini di «**business domain**».

- Ogni funzione deve offrire un valore aziendale reale e misurabile: infatti, se una funzionalità non riconduce ad almeno un obiettivo aziendale, in primo luogo bisognerebbe chiedersi perché la si stia implementando.
- E' anche meno comunemente indicato come **Storytest Driven Development** (STDD)

- Lo sviluppo guidato dal **comportamento** (BDD) combina le tecniche e i principi generali del TDD con le idee tratte del domain-drive design.
- **BDD** è un'attività di **progettazione** in cui si costruiscono pezzi di funzionalità guidati in modo incrementale dal comportamento previsto.
- Il focus di BDD è sul **linguaggio e le interazioni** utilizzate nel processo di sviluppo del software.
- Gli sviluppatori «**Behavior-Driven**» (comportamento) usano il loro linguaggio «**nativo**» in combinazione con la lingua del «**Domain Driven Design**» per descrivere lo scopo e i vantaggi del proprio codice.
- Di solito è definito in un formato **GWT**: GIVEN WHEN & THEN (DATO QUANDO E QUINDI)

- TDD è più un paradigma che un processo.
- Esso descrive prima il ciclo di scrittura di un test e poi quello di implementazione del codice dell'applicazione; il processo è **iterativo** e prevede eventuali attività di refactoring fino al successo di tutti gli unit test.
- TDD però **non risponde** alle seguenti domande:
 - Dove comincio a sviluppare ?
 - Che cosa esattamente dovrei testare ?
 - Come dovrebbero essere strutturati e nominati i test ?
- Quando lo sviluppo è «**Behavior-Driven**» si inizia sempre con le funzionalità più importanti per l'utente.

- TDD e BDD hanno **differenze** linguistiche, i test BDD sono scritti in un linguaggio simile all'inglese.
- BDD si concentra sull'aspetto **comportamentale** del sistema a differenza di TDD che si concentra sull'aspetto dell'**implementazione** del sistema.
- ATDD si concentra sull'**acquisizione dei requisiti** nei test di accettazione e li utilizza per guidare lo sviluppo. (Il sistema fa quello che deve fare?)
- BDD è incentrato sul **cliente**, mentre ATDD fa focus sul come **sviluppare «strutture»** tipo gli Unit TDD.
- Ciò permette una collaborazione molto più facile con gli stakeholder non tecnici, rispetto ai TDD.

- Gli strumenti e le tecniche TDD hanno una natura molto più tecnica e richiedono di acquisire familiarità con il modello di dettaglio ad **oggetti** (o di fatto creare il modello a oggetti nel processo, se si esegue un vero TDD canonico test-first).
- Il tipico stakeholder esecutivo non programmatore si perderebbe completamente nel tentativo di seguire il modello TDD.
- BDD offre una **comprensione più chiara** di ciò che il sistema dovrebbe fare dal punto di vista sia dello sviluppatore sia del cliente.
- TDD consente un design **valido e robusto**, tuttavia, i test possono essere molto lontani dai requisiti degli utenti.
- BDD è un modo per garantire la coerenza tra i requisiti e i test degli sviluppatori.

TOPIC 7

Modellazione funzionale con Data Flow Diagram

- Il Data Flow Diagram (**DFD**) è un tipo di **diagramma** definito nel 1978 da Tom DeMarco nel testo Structured Analysis And Systems Specification per aiutare nella definizione delle specifiche.
- Traggono origine dalla **teoria dei grafi** e sono stati utilizzati anche precedentemente all'avvento dei computer per la gestione delle informazioni.
- È una **notazione grafica** molto usata per i sistemi informativi e per la descrizione del flusso di dati in quanto permette di descrivere un sistema per livelli di astrazione decrescenti con una notazione di specifica molto «intuitiva».
- Non esiste in letteratura a tutt'oggi una definizione universalmente accettata e sono presenti molteplici differenti formulazioni operative.

- Attraverso i Data Flow Diagram si definiscono soprattutto come fluiscono (e vengono elaborate) le informazioni all'interno del sistema, quindi l'oggetto principale è il flusso delle informazioni o, per meglio dire, dei dati.
- Motivo per il quale diventa fondamentale capire **dove** sono immagazzinati i dati, da che fonte provengono, su quale fonte arrivano, quali componenti del sistema li **elaborano**.

- **Processi o Funzioni** (detti anche bolle) che trasformano dati;



- **Flussi** che muovono dati; 

- **Sorgenti esterni** (detti anche terminatori o Agenti Esterni) che producono e consumano dati;



- **Archivi di dati** che memorizzano informazioni in modo passivo.



- Funzioni

- Rappresentano unità di elaborazione dei dati
- Trasformano i dati in ingresso in quelli in uscita

- Flusso di dati

- Le frecce collegano i diversi componenti di un diagramma tra loro
- Rappresentano i dati gestiti dal sistema
- Gli archivi e gli agenti esterni NON possono essere collegati tra loro

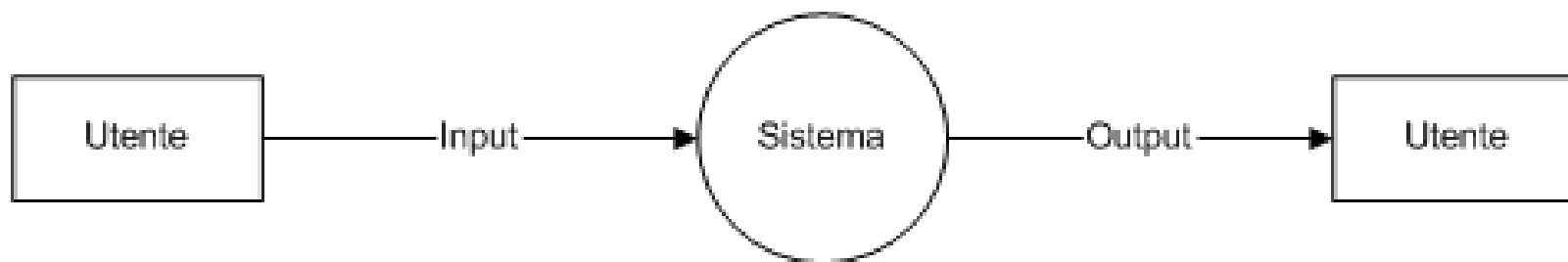
- Archivi

- Sono dei depositi permanenti di informazione
- Possono essere basati su qualunque tecnologia
- I dati che entrano in un archivio vengono scritti
- I dati che escono dall'archivio sono letti (ma non cancellati)

- Agenti esterni

- Rappresentano delle entità esterne al sistema
- Non sono soggette ad ulteriore modellazione
- Sono le sorgenti e le destinazioni dei dati del sistema

- Un generico sistema **è sempre rappresentabile** nel seguente modo



- Se gli ingressi e/o le uscite **sono molteplici** si introducono nuovi flussi.
- Questo tipo di **rappresentazione** ha un livello di astrazione elevato e individua solo l'interfaccia tra il sistema e il mondo esterno per cui vanno inseriti altri dettagli raffinando le funzioni.
- Ogni **funzione**, infatti, è a sua volta specificabile mediante un Data Flow Diagram per cui è possibile ottenere diversi livelli con sempre maggiore definizione.

- un DFD è definibile come una funzione $\langle P, D, A, F \rangle$ dove:

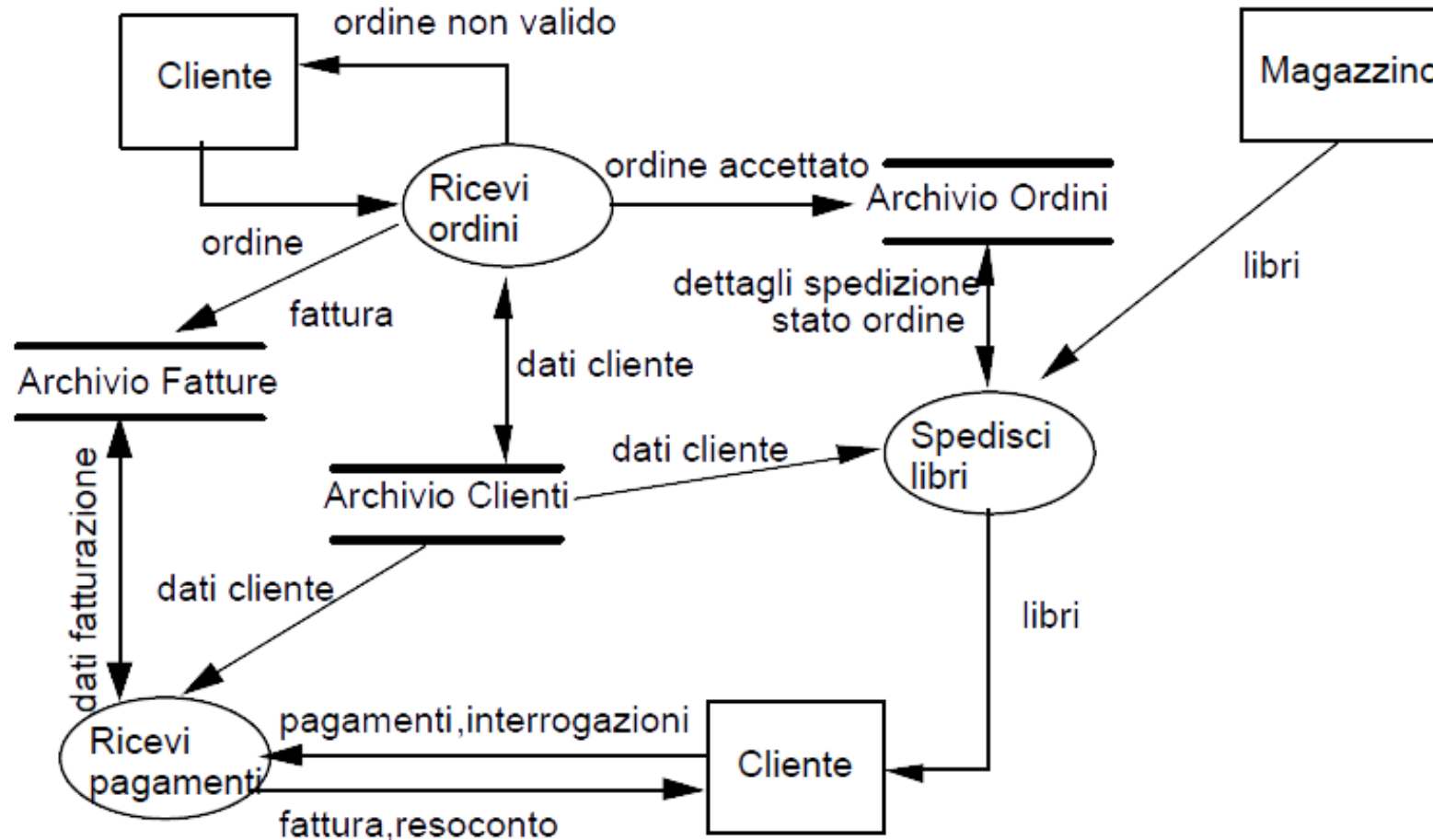
$P = \{p_1, p_2, \dots, p_n\}$ è un insieme finito, non vuoto, di processi;

$D = \{d_1, d_2, \dots, d_r\}$ è un insieme finito di depositi;

$A = \{a_1, a_2, \dots, a_s\}$ è un insieme finito di agenti;

$F = \{ f \in (P \times (P \cup D \cup A)) \cup ((P \cup D \cup A) \times P) \}$ è un insieme finito di flussi.

- Tramite un **grafo orientato** in cui ogni nodo appartiene a uno dei tre insiemi P,D o A, e ogni **arco orientato** rappresenta un flusso di dati. Esempio: Un DFD che descrive i rapporti fra una libreria e i suoi clienti



- Nomi univoci per **identificare** processi, flussi di dati, agenti e depositi.
- Non rappresentare le **eccezioni** e il trattamento degli errori, rimandando questi dettagli alle fasi finali dell'analisi.
- Un DFD **non è un diagramma di flusso**, dove le frecce indicano un ordinamento negli eventi.
- In alcune estensioni, come nella notazione OMT è possibile rappresentare anche flussi di controllo; anticiparli in un DFD comporta di fatto una duplicazione se si usa anche un modello dinamico.

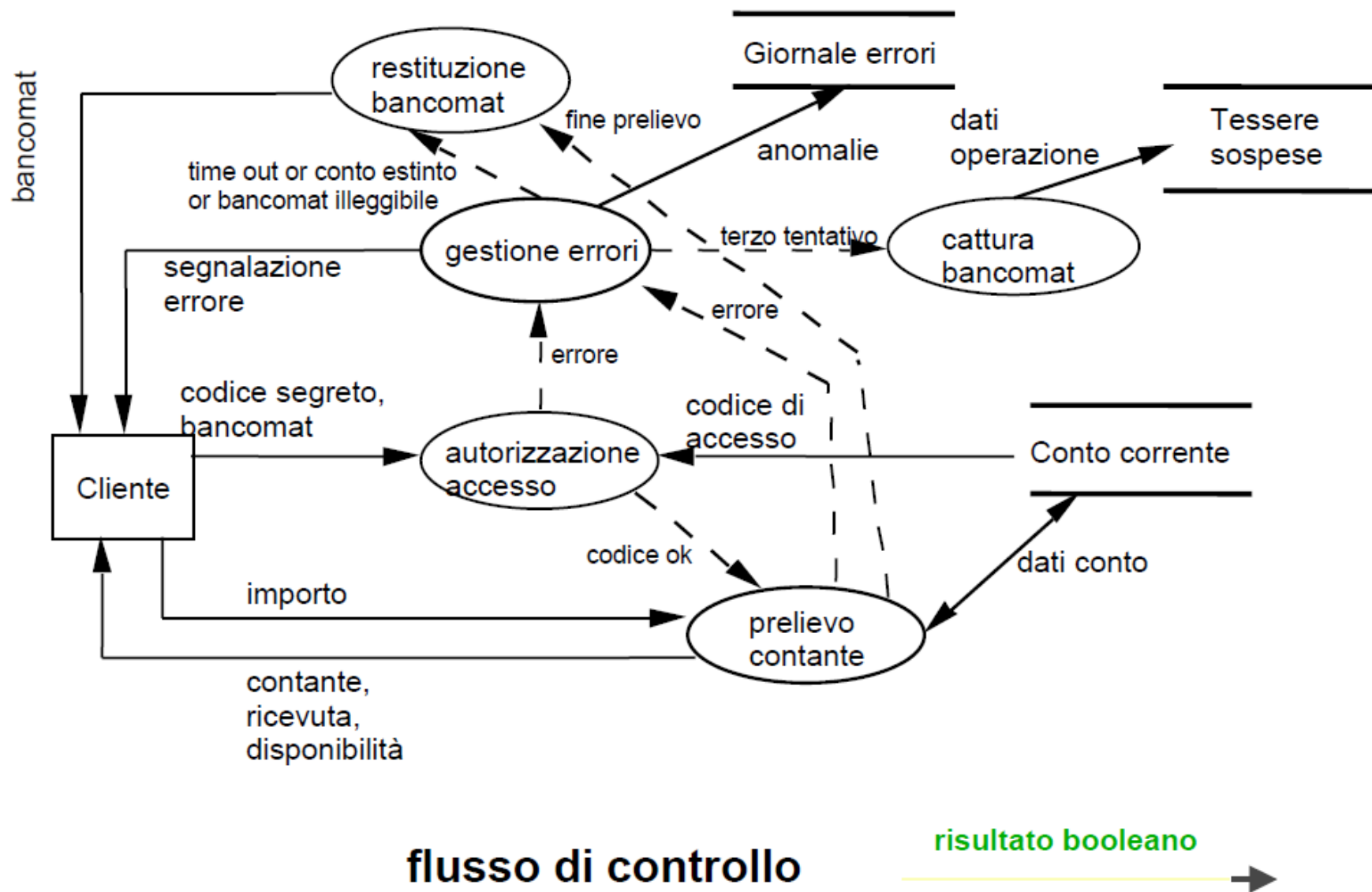
- Scegliere **nomi significativi** per i processi, flussi, depositi, e agenti.
- **Numerare** i processi.
- **Disegnare** i DFD seguendo criteri estetici.
- **Evitare** DFD eccessivamente complessi.
- **Accertarsi della coerenza interna** di un DFD e che sia coerente con quelli ad esso associati.

- **Evitare** processi a consumo infinito (pozzi).
- **Sospettare** di processi a generazione spontanea.
- **Depositi** a sola lettura o a sola scrittura sono rari.
- **Non devono esistere** flussi di dati fra :
 - due agenti esterni,
 - due depositi,
 - un'entità esterna e un deposito

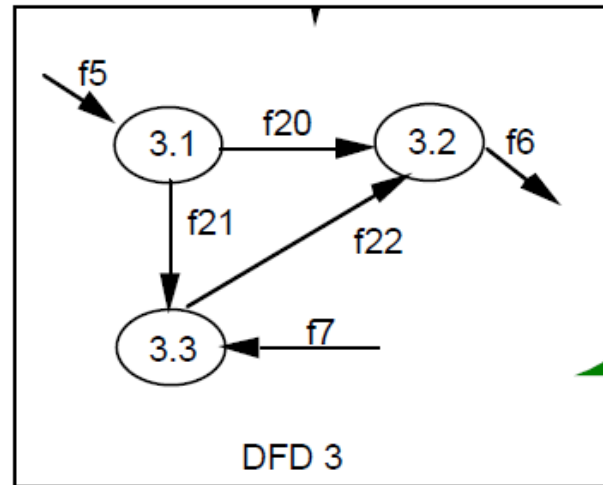
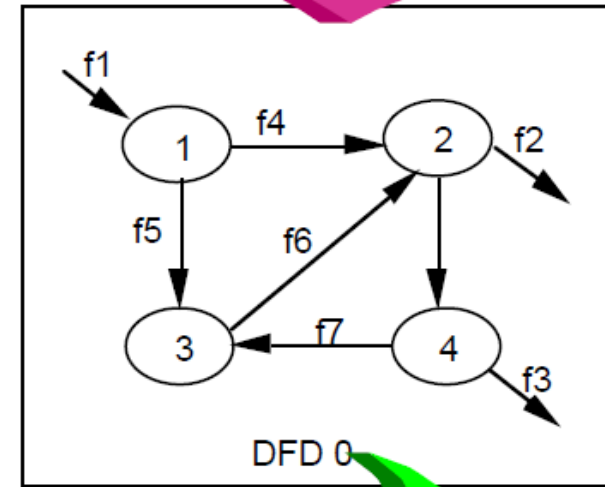
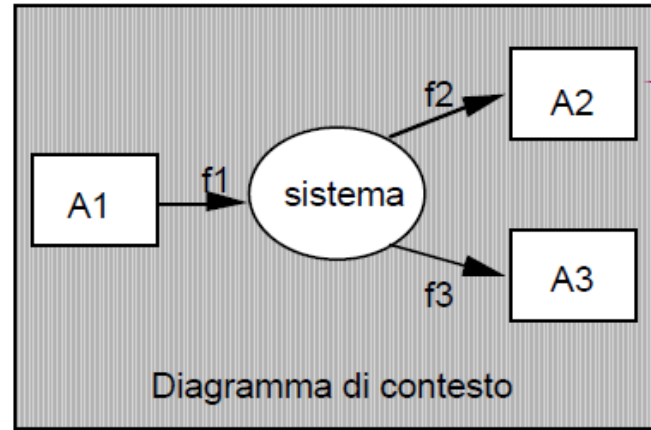
- L'assenza di nomi per flussi, processi e depositi può essere indice di trascuratezza, di indecisione ma può nascondere anche insidie **più gravi**, ad esempio che l'analista confonde tra un flow-chart e un DFD.
- C'è una **convenzione**, spesso accettata in molte versioni dei DFD, che un flusso da o verso un deposito possa essere non etichettato quando i dati trasferiti corrispondono ad un oggetto (record) intero.



DFD con flussi di controllo

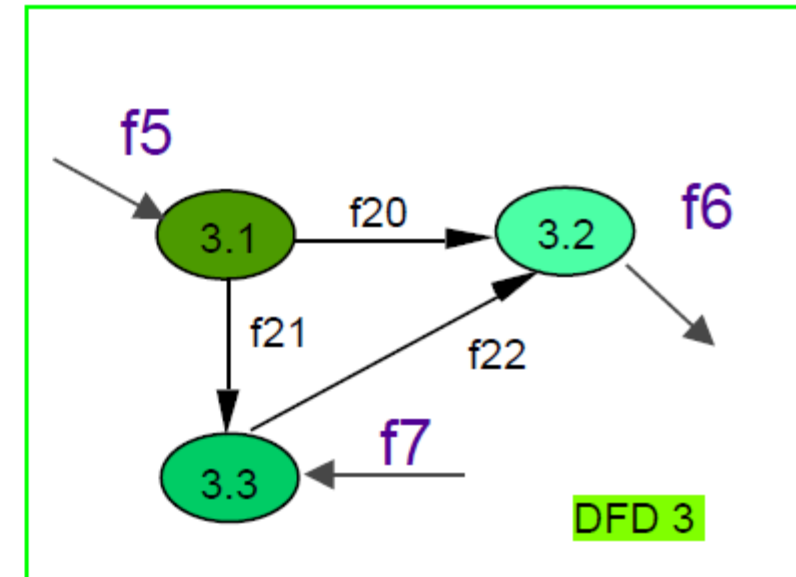
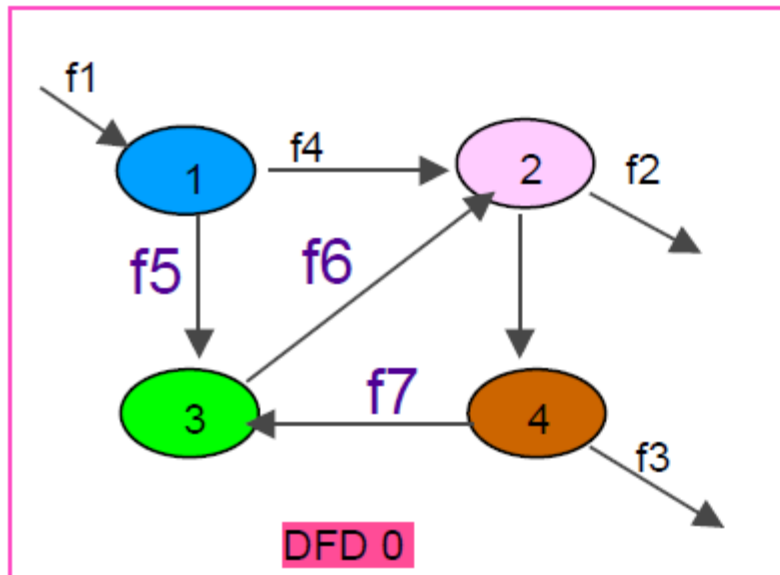


- Non è sensato pensare di sviluppare un singolo DFD per modellare con sufficiente dettaglio un ambiente reale.
- Un'applicazione di medie dimensioni richiede da **tre a sei livelli**.
- **Non esiste una ricetta** per dire quanti livelli sono necessari per modellare una certa realtà e anche qui valgono considerazioni dettate dal buon senso.



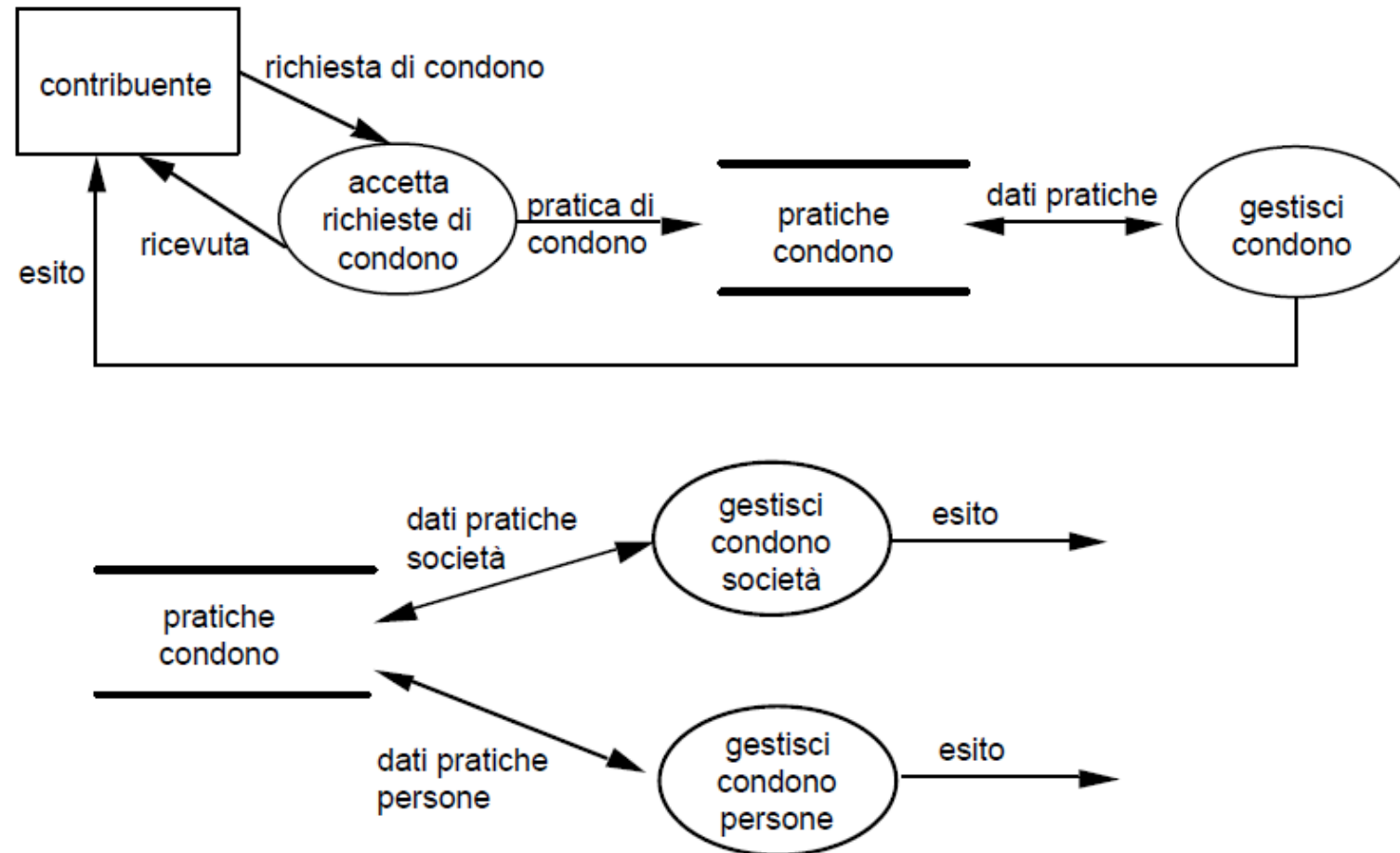
- Evitare di costruire DFD molto sbilanciati
 - Se in un DFD compaiono bolle atomiche e bolle che richiedono successivi livelli di dettaglio ciò è sintomo di **trascuratezza nel modellare il sistema**.
- Prestare attenzione al problema della presentazione dei DFD
 - Può essere molto utile affiancare a documenti cartacei **strumenti di visualizzazione** che consentano di navigare la gerarchia mostrandone viste a vari livelli di astrazione

- Verificare che i DFD siano fra loro coerenti
 - Al fine di **garantire** che ciascun DFD sia coerente con il DFD genitore si verifichi che i flussi di dati relativi a una bolla a un livello **corrispondano** ai flussi di dati evidenziati nel DFD che esplode quella bolla.

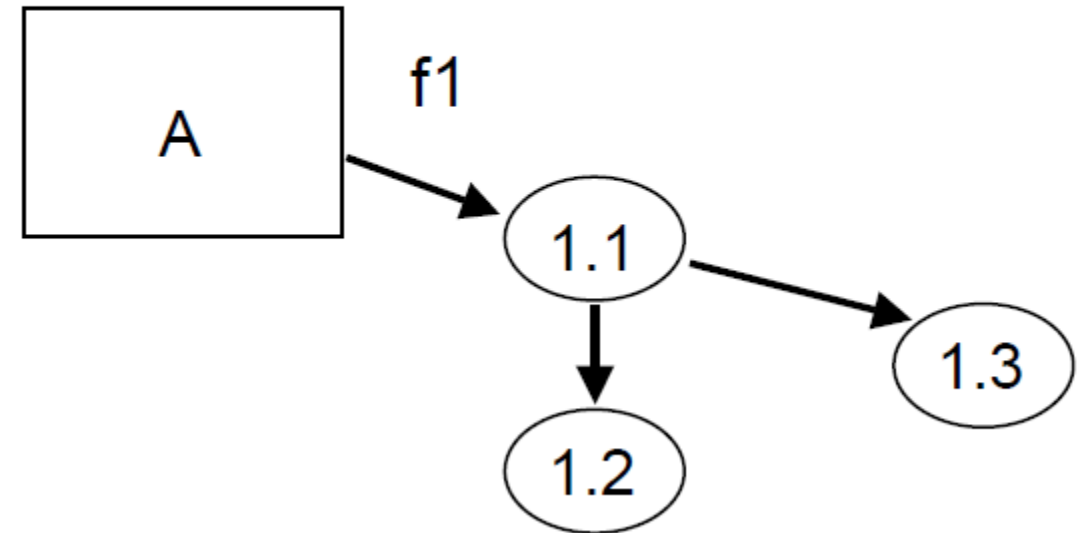
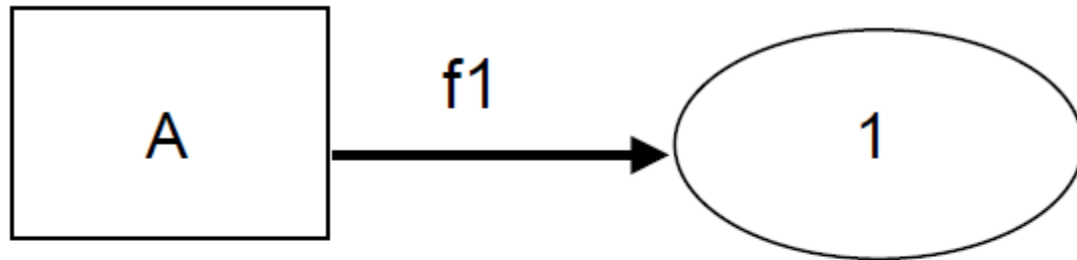


Consigli pratici

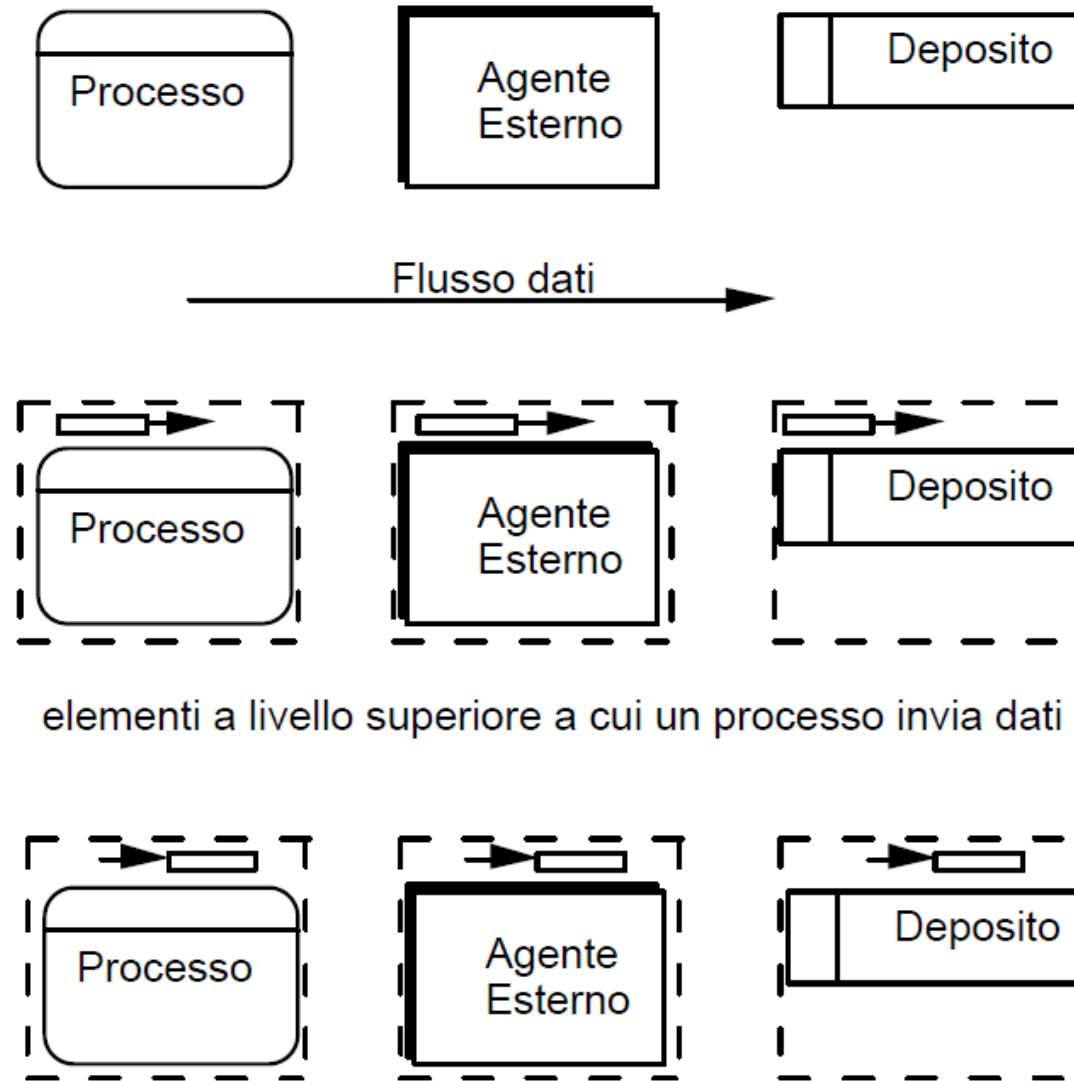
- Mostrare un deposito al livello più alto di astrazione, quando se ne scopre la necessità come interfaccia fra due o più processi, quindi riportare il deposito in ogni diagramma di livello inferiore che descrive quei processi.



- Verificare che un agente esterno collegato a un processo in un certo livello compaia e resti connesso a un discendente di quel processo nel livello gerarchico inferiore.



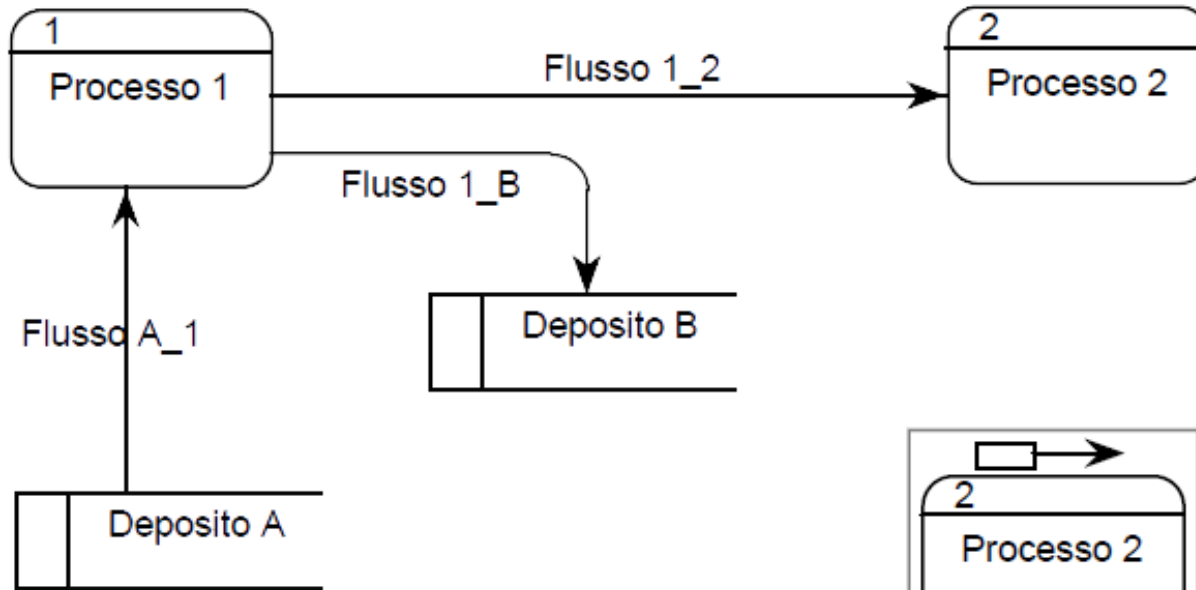
- **Top-down**: decomposizione di un processo in una serie di sottoprocessi chiaramente identificabili e indipendenti.
- **Bottom-up**: a partire da una collezione di concetti elementari si costruiscono via via le connessioni fra essi.
- **Mixed**: raffinamento di un DFD di massima in stadi successivi con tecniche top-down e bottom-up
- **Outside-in**: parte dalle interfacce con il sistema e propaga in avanti gli ingressi evidenziando i processi coinvolti nei flussi di dati o, in alternativa, propaga all'indietro le uscite.



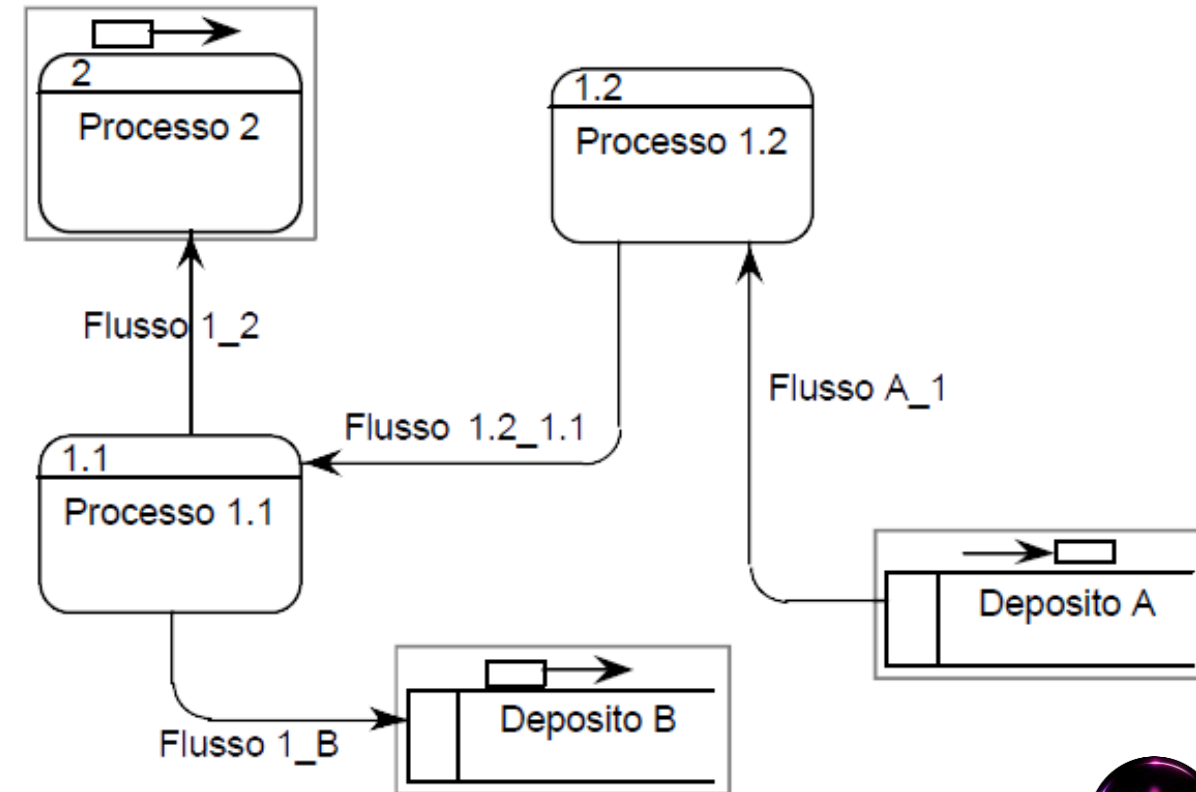
elementi a livello superiore a cui un processo invia dati

elementi a livello superiore da cui un processo riceve dati

Un DFD



Esplosione del Processo 1



- Nella stesura **si ignora l'inizializzazione del sistema**, la gestione degli errori e la terminazione, il sistema si immagina come «up & running».
- Si ignorano anche le **sincronizzazioni** ed il **flusso di controllo** tra processi.
- Individuare sempre le entrate e le uscite di un diagramma.
- Qualora i dati gestiti fossero particolarmente strutturati, si affianca al Data Flow Diagram un sistema complementare.

- Questa notazione presenta dei limiti consistenti:
 - **Semantica:** i simboli non sono sufficientemente chiari e i nomi vengono scelti dall'utente
 - **Controllo:** gli aspetti di controllo non sono definiti dal modello e, quindi, anche la sequenza temporale è poco chiara.
- Il DFD è adatto ad una descrizione rapida e intuitiva per cui **non è una notazione operativa** proprio perché alcuni aspetti non sono chiariti; per questo motivo si parla di notazione semi-formale perché la sintassi è precisa, ma la semantica non lo è.
- Sono stati progettati diversi metodi per rimediare a queste difficoltà che possono essere classificati nel seguente modo:
 - Usare una **notazione complementare** per colmare le lacune del data flow diagram
 - **Migliorare il modello** in modo da completare la versione tradizionale